ACTIVIDAD EVALUATIVA

EJE 2 - Entrega 2: Desarrollo inicial del sistema de votaciones

Sistema Web de Votaciones – Festival de la Leyenda Vallenata 2027

SEBASTIÁN BAUTISTA MARTÍNEZ
MARÍA LABARCA BRICEÑO
JUAN MEJÍA MAESTRE

PROFESOR

DEIVYS MORALES URIBE

DESARROLLO WEB - 4324 - 61
[PROGRAMA ACADÉMICO]
FUNDACIÓN UNIVERSITARIA DEL ÁREA ANDINA

OCTUBRE DE 2026

AREANDINA

Tabla de Contenidos

Introducción y objetivo 3
Integrantes y distribución del trabajo 4
Desarrollo técnico 5
- Tecnologías 5
- Estructura del proyecto Django 5
- Patrón de arquitectura 5
- Conexión con MySQL 6
- Modelos y relaciones 7
- Migraciones 10
- Autenticación 10
- Administración: panel propio y Django Admin 11

Capturas de pantalla comentadas 12
- Django Admin 12
- Vistas y documentación de la API 17
- Interfaz pública con datos de MySQL 18
- Autenticación 21
- Panel de administración propio (CRUD con Django) 25
- Visualización dinámica 29

Pruebas y resultados 30
- Pruebas automatizadas 30
- Pruebas funcionales 31

Repositorio 32
Video demostrativo 33
Conclusiones y distribución del trabajo 34
- Aprendizajes 34
- Dificultades 34
- Pendientes para la siguiente entrega 34
- Distribución del trabajo 34

Referencias 36

AREANDINA

Introducción y objetivo

El presente documento corresponde a la Entrega 2 del proyecto de la asignatura Desarrollo Web: el desarrollo inicial del Sistema Web de Votaciones para el Festival de la Leyenda Vallenata 2027. A partir del análisis y del diseño de la Entrega 1 (actores, requerimientos, reglas de negocio, casos de uso y modelo conceptual), el equipo construyó la aplicación con Python, Django (patrón MVT) y MySQL: vistas y plantillas de Django que muestran los datos de la base de datos, registro, inicio y cierre de sesión con el sistema de autenticación de Django, y un panel de administración propio para ediciones, categorías, votaciones y opciones, además de Django Admin.

El modelo de datos respeta la jerarquía definida en la primera entrega, Edición → Categoría → Votación → Opción, y queda preparado para incorporar el voto y sus restricciones en la siguiente entrega. Como la plataforma es configurable, ninguna categoría ni votación está escrita en el código: todas se crean desde el panel y las vistas de Django las leen de MySQL.

Objetivo de la entrega. Implementar la estructura inicial de la aplicación web con Python, Django y MySQL, integrando las primeras funcionalidades del sistema de votaciones a partir del análisis y diseño de la primera entrega.

Tabla 1

Funcionalidades mínimas de la Entrega 2 y dónde se evidencian

Funcionalidad exigida	Estado	Evidencia
Proyecto Django: aplicaciones, rutas, vistas y plantillas	Cumplido	Figuras 1 a 3 y 9; Figuras 21 a 37
MySQL: conexión, modelos relacionados y migraciones	Cumplido	Figuras 4, 5 y 8; Tabla 4
Autenticación: registro, inicio y cierre de sesión	Cumplido	Figuras 9 y 25 a 29
Ediciones: registrar y consultar ediciones anuales	Cumplido	Figuras 13 y 31
Categorías vinculadas a una edición	Cumplido	Figuras 14, 17, 18, 22 y 32 a 34
Votaciones y opciones	Cumplido	Figuras 15, 16, 24 y 35 a 37
Interfaz pública con datos reales de MySQL (vistas y plantillas Django)	Cumplido	Figuras 21 a 24 y 38

Integrantes y distribución del trabajo

El equipo está conformado por tres integrantes. Cada uno registra sus aportes con su propia cuenta en el repositorio de GitHub, lo que deja trazabilidad de quién hizo cada parte. La distribución detallada de responsabilidades se presenta en la sección de conclusiones (Tabla 7).

Tabla 2

Integrantes del equipo

Integrante	Usuario de GitHub	Rol principal
Juan Mejía Maestre	juandiegows	Coordinación, frontend y despliegue
Sebastián Bautista Martínez	sbautista15	Backend Django, modelos y base de datos
María Labarca Briceño	mlabarca-jpg	Documentación, pruebas y administración

Nota. La actividad indica grupos de cuatro estudiantes; este equipo está conformado por tres integrantes, por lo que cada uno asume más de un frente de trabajo.

Desarrollo técnico

Tecnologías

Tabla 3

Tecnologías utilizadas

Capa	Tecnología	Uso en el proyecto
Backend	Python 3.12, Django 5.2	Modelos, migraciones, autenticación, vistas y plantillas (MVT), Django Admin y reglas de negocio
Interfaz (MVT)	Plantillas de Django, HTML5, CSS3, Bootstrap 5.3, Bootstrap Icons y JavaScript	Sitio público, registro e inicio de sesión y panel de administración propio
Base de datos	MySQL 8.4 (conector PyMySQL)	Persistencia de todas las entidades mediante el ORM de Django
API REST	Django REST Framework 3.18	Las mismas reglas expuestas en JSON para otros clientes (la web React del repositorio)
Correo	ZeptoMail (Zoho) o SMTP	Confirmación del correo y bienvenida, con adaptadores intercambiables
Entorno	Docker Compose, Git y GitHub	MySQL y Django en contenedores; control de versiones e integración continua

Estructura del proyecto Django

El backend vive en app/api. El proyecto se llama config (configuración, rutas raíz, adaptador de base de datos y servidor WSGI) y tiene dos aplicaciones: portal, que contiene las vistas, formularios y plantillas del sitio (patrón MVT), y votaciones, que agrupa por dominio los modelos, las reglas de negocio, la API REST y Django Admin. Las dos comparten los mismos modelos y servicios, por lo que ninguna regla se escribe dos veces.

Figura 1

Estructura de carpetas del backend

```
app/api/
app/api/
├── config/              settings.py, urls.py, base_datos.py (adaptadores MySQL y SQLite)
├── portal/              aplicación del sitio (MVT)
│   ├── vistas/          publico.py, cuentas.py y panel/ (un módulo por entidad)
│   ├── templates/       base.html, parciales/ y panel/
│   ├── formularios.py   registro, inicio de sesión y CRUD del panel
│   ├── static/          Bootstrap 5, Bootstrap Icons y hojas de estilo de la marca
│   └── tests/           pruebas del sitio, las cuentas y el panel
├── votaciones/          dominio: modelos, servicios, selectores, API REST y Django Admin
│   ├── cuentas/         Usuario: registro, confirmación del correo
│   ├── votacion/        Edición → Categoría → Votación → Opción → Voto
│   ├── correo/          adaptadores de correo: ZeptoMail, SMTP, consola y memoria
│   ├── auditoria/ · sitio/ · comun/
│   └── migrations/      0001_initial, 0002_pedir_documento
├── Dockerfile · entrypoint.sh (espera a MySQL, ejecuta migrate y arranca Gunicorn)
└── requirements.txt
```

Nota. Elaboración propia a partir del repositorio. Se resumen las carpetas principales.

Patrón de arquitectura

Django organiza la aplicación en Modelo, Vista y Plantilla (MVT), y así quedó construida

esta entrega:

• Modelo: las clases de votaciones/votacion/models.py definen las tablas de MySQL y sus relaciones; las consultas se hacen con el ORM de Django, sin escribir SQL.

• Vista y plantilla: las funciones de portal/vistas/ reciben cada petición, consultan los modelos y entregan los datos a una plantilla de templates/portal/, que genera el HTML con el diseño de la marca (Bootstrap y las hojas de estilo propias). El panel /panel usa el mismo patrón con formularios de Django.

Las reglas de negocio están en una capa de servicios (servicios.py) y las consultas en selectores, de modo que las vistas del sitio, la API REST y Django Admin usan las mismas reglas. La base de datos y el correo se conectan con adaptadores intercambiables (patrón puerto/adaptador). El flujo de una petición es: navegador → urls.py → vista de Django → servicio o selector → modelo (ORM) → MySQL → plantilla → HTML.

Figura 2

Rutas raíz del proyecto

app/api/config/urls.py

```
urlpatterns = [
    path("django-admin/", admin.site.urls),        # Django Admin
    path("api/", include("votaciones.urls")),       # API REST
    path("", include("portal.urls")),               # sitio Django (MVT)
]
```

Nota. Fragmento del repositorio (app/api/config/urls.py). Se omiten líneas que no son necesarias para la explicación.

Figura 3

Rutas del sitio: autenticación, panel y páginas públicas

app/api/portal/urls.py

```
path("registro", cuentas.registro, name="registro"),
path("login", cuentas.iniciar_sesion, name="login"),
path("logout", cuentas.cerrar_sesion, name="logout"),
path("panel/categorias", categorias.categorias, name="panel-categorias"),
path("<anio:anio>", publico.edicion, name="edicion"),
path("<anio:anio>/categorias/<slug:categoria>", publico.categoria, name="categoria"),
```

Nota. Fragmento del repositorio (app/api/portal/urls.py). Se omiten líneas que no son necesarias para la explicación.

Conexión con MySQL

La conexión se configura con un adaptador (config/base_datos.py): la variable DB_ENGINE=mysql activa AdaptadorMySQL, que usa el conector PyMySQL, la codificación utf8mb4 (tildes y ñ), el modo estricto y conexiones persistentes. Los datos de acceso se leen de variables de entorno, de modo que ninguna contraseña queda escrita en el código. Para desarrollo rápido sin Docker, AdaptadorSQLite usa un archivo local; el resto del sistema solo conoce el ORM.

Figura 4

Adaptador de base de datos para MySQL

app/api/config/base_datos.py

```
class AdaptadorMySQL(AdaptadorBaseDatos):
    def preparar(self):
        import pymysql
        pymysql.install_as_MySQLdb()

    def configuracion(self, base_dir):
        return {
            "ENGINE": "django.db.backends.mysql",
            "NAME": os.getenv("DB_NAME", "votaciones_flv"),
            "USER": os.getenv("DB_USER", "votaciones"),
            "PASSWORD": os.getenv("DB_PASSWORD", ""),
            "HOST": os.getenv("DB_HOST", "127.0.0.1"),
            "CONN_MAX_AGE": 60, "CONN_HEALTH_CHECKS": True,
            "OPTIONS": {"charset": "utf8mb4",
                        "init_command": "SET sql_mode='STRICT_TRANS_TABLES'"},
        }

# config/settings.py
DATABASES = {"default": base_datos.configurar(os.getenv("DB_ENGINE", "sqlite"), BASE_DIR)}
```

Nota. Fragmento del repositorio (app/api/config/base_datos.py). Los valores reales de usuario y contraseña se entregan al contenedor como variables de entorno y no se incluyen en este documento.

Figura 5

Versión de MySQL, tablas creadas por las migraciones y datos de prueba por edición

docker compose exec db mysql … votaciones_flv

```
VERSION()
8.4.11
Tables_in_votaciones_flv
auth_group
auth_group_permissions
auth_permission
authtoken_token
banner_inicio
categoria
configuracion_sitio
django_admin_log
django_content_type
django_migrations
django_session
edicion
opcion
red_social
registro_auditoria
revista
usuario
usuario_groups
usuario_user_permissions
votacion
voto
anio  categorias  votaciones  opciones
2025  2  3  8
2026  2  2  7
2027  6  8  32
```

Nota. Salida de consola tomada el 18 de octubre de 2026 en el entorno Docker local. La edición 2027 incluye la categoría creada durante la prueba de la Figura 17.

Modelos y relaciones

Los modelos implementan la jerarquía del modelo conceptual de la Entrega 1. Cada relación

es una llave foránea con `on_delete=PROTECT`, que impide borrar una edición con categorías o una votación con opciones y votos. Las reglas de integridad se declaran como restricciones de la base de datos (por ejemplo, la fecha de cierre debe ser posterior a la de apertura y el nombre de la categoría no se repite dentro de la misma edición).

Tabla 4

Modelos principales, relaciones y cardinalidades

Modelo (tabla)	**Campos principales**	**Relación**
Edicion (`edicion`)	nombre, anio (único), fecha_inicio, fecha_fin, estado, votos_por_usuario, visibilidad_resultados	1 edición → N categorías
Categoria (`categoria`)	edicion, nombre, slug, descripcion, icono, activa, orden	N categorías → 1 edición; 1 categoría → N votaciones
Votacion (`votacion`)	categoria, titulo, slug, fecha_apertura, fecha_cierre, votos_por_usuario, publicada	N votaciones → 1 categoría; 1 votación → N opciones
Opcion (`opcion`)	votacion, nombre, descripcion, imagen, enlace_multimedia, audio, orden, activa	N opciones → 1 votación
Usuario (`usuario`)	email (único, se usa para iniciar sesión), nombres, apellidos, rol, correo_verificado	1 usuario → N votos
Voto (`voto`)	usuario, votacion, opcion, fecha_hora, codigo_comprobante (único)	Preparado para la Entrega 3: une usuario, votación y opción

Nota. Todos los modelos heredan de ModeloTrazable, que agrega creado_en, creado_por, actualizado_en y actualizado_por.

Figura 6

Modelos Edición y Categoría (relación uno a muchos)

app/api/votaciones/votacion/models.py

```
class Edicion(ModeloTrazable):
    nombre = models.CharField(max_length=150)
    anio = models.PositiveSmallIntegerField("año", unique=True)
    fecha_inicio = models.DateField()
    fecha_fin = models.DateField()
    estado = models.CharField(max_length=10, choices=Estado.choices, default=Estado.ACTIVA)
    votos_por_usuario = models.PositiveSmallIntegerField(default=1)


    class Meta:
        db_table = "edicion"
        constraints = [models.CheckConstraint(
            condition=Q(fecha_fin__gte=F("fecha_inicio")), name="edicion_fechas_validas")]



class Categoria(ModeloTrazable):
    edicion = models.ForeignKey(Edicion, on_delete=models.PROTECT,
related_name="categorias")
    nombre = models.CharField(max_length=120)
    slug = models.SlugField(max_length=120, blank=True)
    activa = models.BooleanField(default=True)
    orden = models.PositiveSmallIntegerField(default=0)


    class Meta:
        db_table = "categoria"
        constraints = [models.UniqueConstraint(
            fields=["edicion", "nombre"], name="categoria_nombre_unico_por_edicion")]
```

Nota. Fragmento del repositorio (app/api/votaciones/votacion/models.py). Se omiten líneas que no son necesarias para la explicación.

Figura 7

Modelos Votación y Opción

app/api/votaciones/votacion/models.py

```
class Votacion(ModeloTrazable):
    categoria = models.ForeignKey(Categoria, on_delete=models.PROTECT,
related_name="votaciones")
    titulo = models.CharField(max_length=150)
    fecha_apertura = models.DateTimeField()
    fecha_cierre = models.DateTimeField()
    votos_por_usuario = models.PositiveSmallIntegerField(default=1)
    publicada = models.BooleanField(default=False)


    class Meta:
        db_table = "votacion"
        constraints = [models.CheckConstraint(
            condition=Q(fecha_cierre__gt=F("fecha_apertura")),
            name="votacion_cierre_posterior_apertura")]



class Opcion(ModeloTrazable):
    votacion = models.ForeignKey(Votacion, on_delete=models.PROTECT,
related_name="opciones")
    nombre = models.CharField(max_length=150)
    descripcion = models.TextField(blank=True)
    orden = models.PositiveSmallIntegerField(default=0)
    activa = models.BooleanField(default=True)
```

Nota. Fragmento del repositorio (app/api/votaciones/votacion/models.py). Se omiten líneas que no son necesarias para la explicación.

Migraciones

Las tablas se crean con el sistema de migraciones de Django. Al arrancar, el contenedor de la API espera a que MySQL esté disponible y ejecuta `python manage.py migrate` (archivo `entrypoint.sh`). En la integración continua de GitHub se ejecuta `makemigrations --check` para comprobar que ningún cambio en los modelos quedó sin su migración.

Figura 8

Migraciones aplicadas en MySQL

```
python manage.py showmigrations votaciones
votaciones
 [X] 0001_initial
 [X] 0002_pedir_documento
```

Nota. Salida de consola tomada el 18 de octubre de 2026 en el entorno Docker local con MySQL 8.4.

Autenticación

La autenticación usa el sistema de usuarios de Django con un modelo propio, Usuario, que hereda de AbstractUser y se declara en AUTH_USER_MODEL. El correo electrónico reemplaza al nombre de usuario para iniciar sesión, las contraseñas se guardan con el hash PBKDF2 de Django y se validan con AUTH_PASSWORD_VALIDATORS. En el sitio, el registro usa un formulario de Django (RegistroForm) y el inicio y cierre de sesión usan authenticate(), login() y logout(), que guardan la sesión en una cookie protegida; el cierre de sesión solo se acepta por POST con token CSRF. El panel exige sesión de administrador mediante el decorador solo_administrador.

Figura 9

Vistas de registro, inicio y cierre de sesión del sitio

```
app/api/portal/vistas/cuentas.py
def registro(request):
    form = RegistroForm(request.POST or None, pedir_documento=pedir_documento)
    if request.method == "POST" and form.is_valid():
        usuario = servicios.registrar_votante(**form.datos_cuenta())
        login(request, usuario, backend="django.contrib.auth.backends.ModelBackend")
        return redirect(destino_seguro(request, reverse("portal:categorias")))
    return render(request, "portal/registro.html", {"form": form, ...})

def iniciar_sesion(request):
    form = LoginForm(request.POST or None, request=request)   # usa authenticate()
    if request.method == "POST" and form.is_valid():
        login(request, form.usuario)
        return redirect(destino_seguro(request, por_defecto))
    return render(request, "portal/login.html", {"form": form, ...})

@require_POST
def cerrar_sesion(request):
    logout(request)
    return redirect("portal:inicio")
```

Nota. Fragmento del repositorio (app/api/portal/vistas/cuentas.py). Se omiten líneas que no son necesarias para la explicación.

Administración: panel propio y Django Admin

Las ediciones, categorías, votaciones y opciones se administran en un panel propio construido con vistas, formularios y plantillas de Django (/panel), con el diseño de la marca: listados con búsqueda y filtros, formularios en ventana modal, activar o desactivar, publicar y cerrar votaciones. Las reglas de negocio se aplican igual que en la API: no se publica una votación con menos de dos opciones, no se borra lo que tiene votos y cada cambio queda en la auditoría. Django Admin sigue disponible como respaldo (Figuras 11 a 19).

Figura 10

Registro de los modelos en Django Admin

app/api/votaciones/votacion/admin.py

```
class OpcionInline(admin.TabularInline):
    model = Opcion
    extra = 0


@admin.register(Edicion)
class EdicionAdmin(TrazableAdmin):
    list_display = ["nombre", "anio", "fecha_inicio", "fecha_fin", "estado"]


@admin.register(Categoria)
class CategoriaAdmin(TrazableAdmin):
    list_display = ["nombre", "edicion", "activa", "orden"]
    list_filter = ["edicion", "activa"]


@admin.register(Votacion)
class VotacionAdmin(TrazableAdmin):
    list_display = ["titulo", "categoria", "fecha_apertura", "fecha_cierre", "publicada",
"estado"]
    list_filter = ["categoria__edicion", "categoria", "publicada"]
    inlines = [OpcionInline]                        # opciones dentro de la votación


@admin.register(Voto)
class VotoAdmin(admin.ModelAdmin):
    def has_change_permission(self, request, obj=None):
        return False                                 # los votos no se modifican
```

Nota. Fragmento del repositorio (app/api/votaciones/votacion/admin.py). Se omiten líneas que no son necesarias para la explicación.

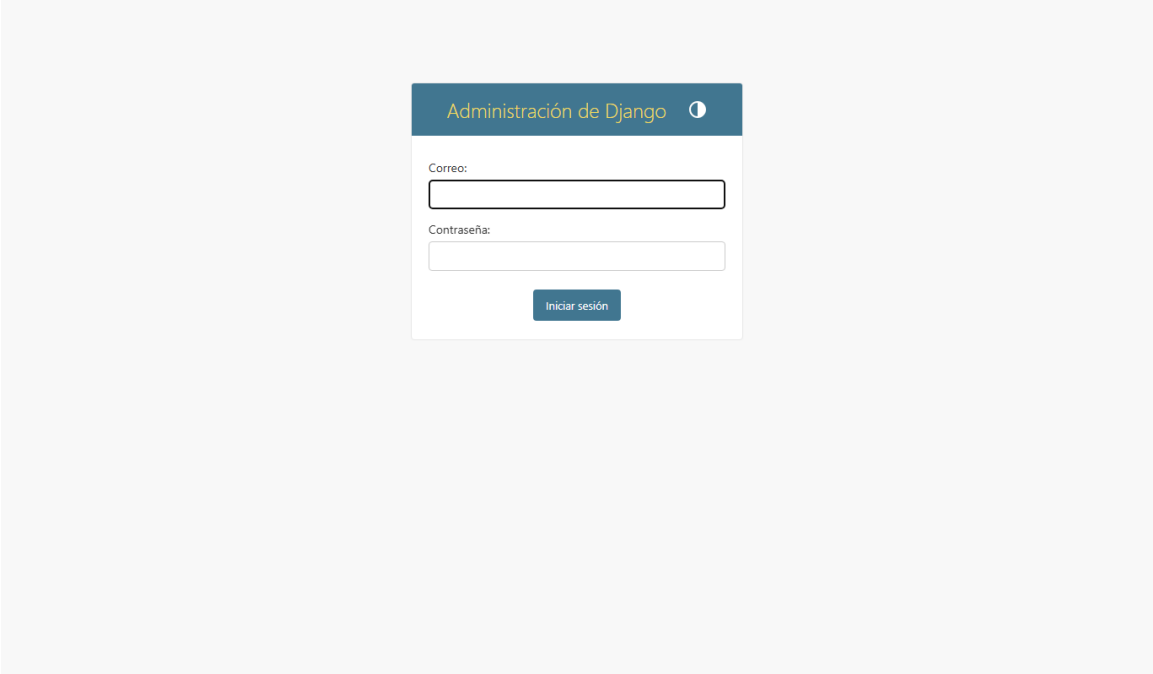
Capturas de pantalla comentadas

Las capturas de Django Admin y de la API se tomaron el 18 de octubre de 2026, y las del sitio y del panel construidos con plantillas de Django, el 20 de octubre de 2026, en el entorno Docker local (MySQL 8.4 y Django), con datos de prueba ficticios. Se usaron un votante y una categoría temporales que se borraron al terminar; ninguna captura muestra contraseñas.

Django Admin

Figura 11

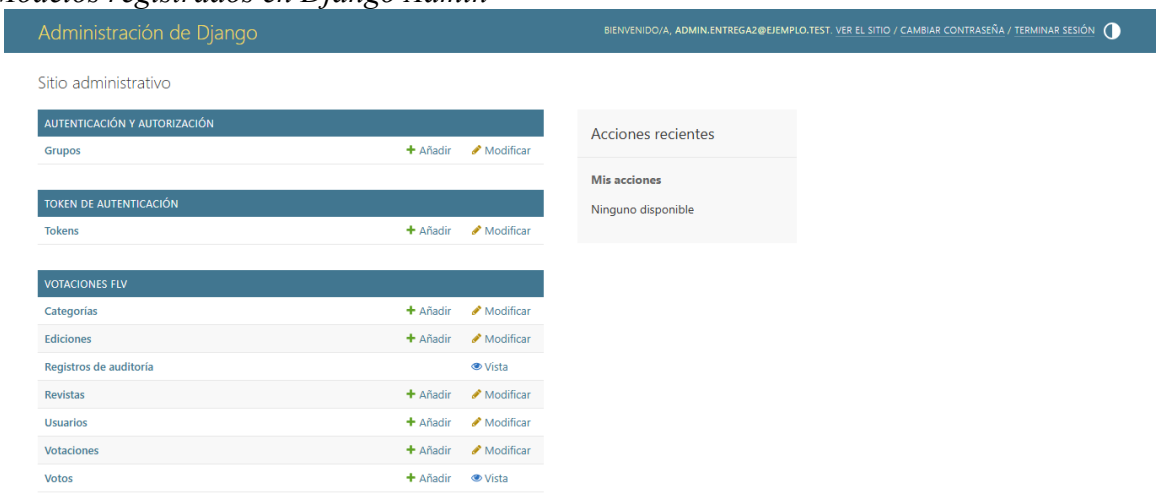
Inicio de sesión en Django Admin



Nota. Solo los usuarios con permiso de personal (is_staff) pueden entrar. El campo de usuario es el correo electrónico, porque el modelo Usuario lo define como USERNAME_FIELD.

Figura 12

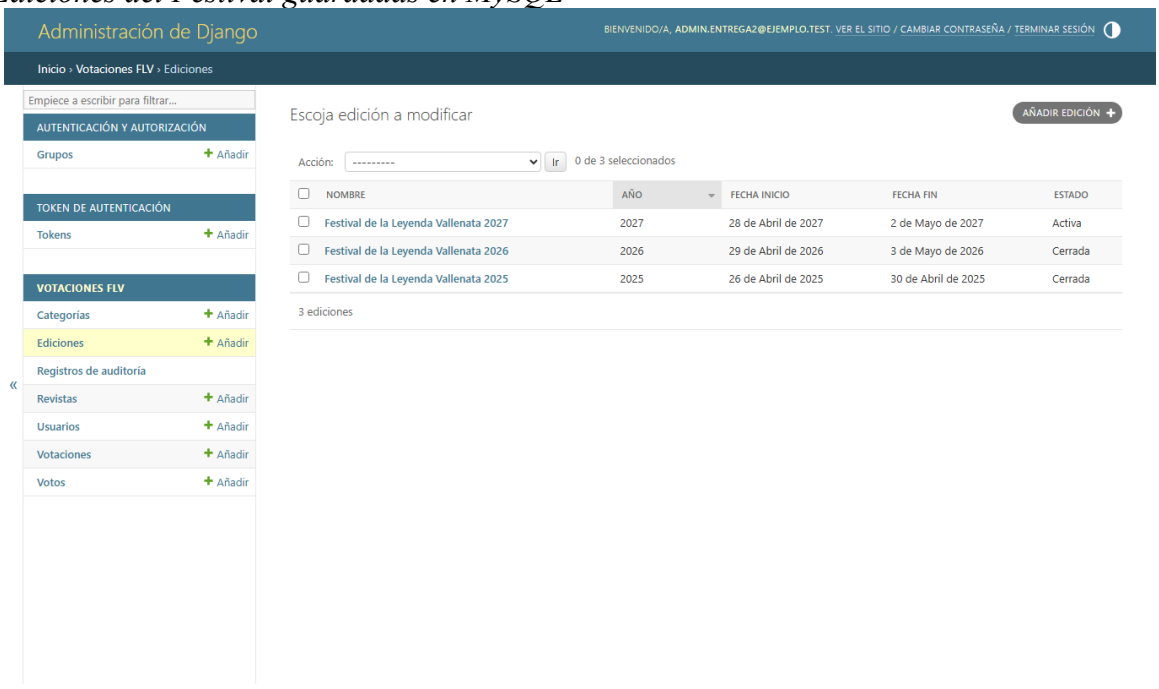
Modelos registrados en Django Admin



Nota. Demuestra que los modelos de la aplicación (Ediciones, Categorías, Votaciones, Usuarios, Votos, Revistas y Registros de auditoría) quedaron disponibles para administrarse. Votos y Auditoría solo permiten consulta.

Figura 13

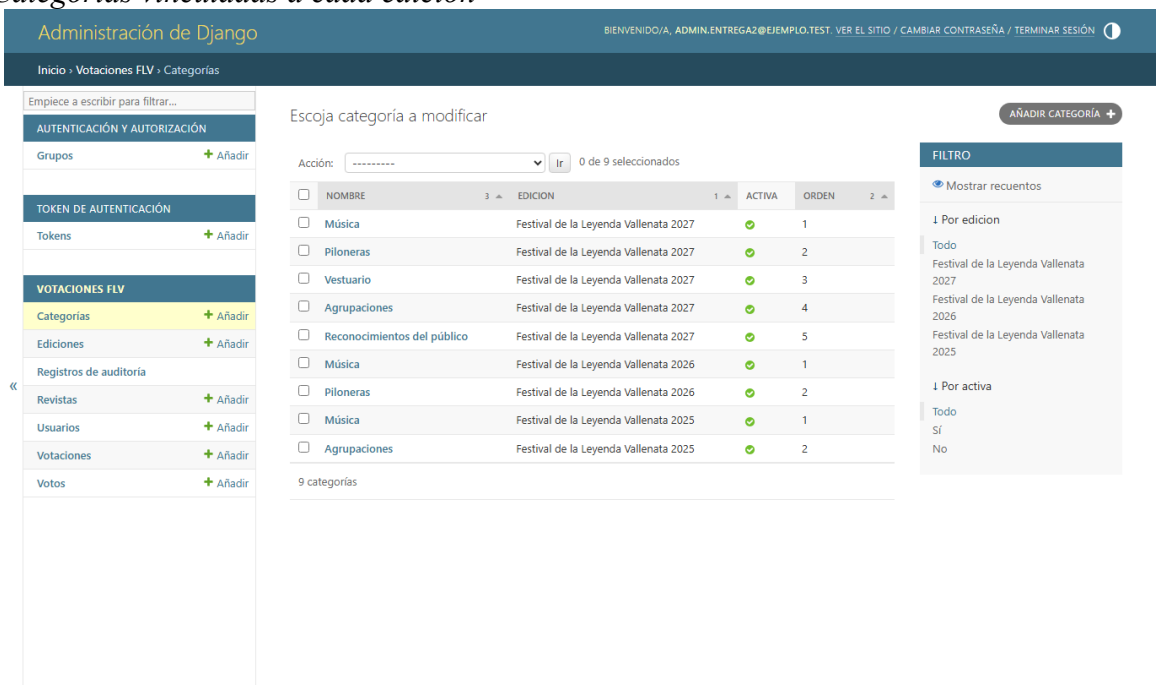
Ediciones del Festival guardadas en MySQL



Nota. Tres ediciones anuales (2025, 2026 y 2027) con sus fechas y estado; la 2027 es la edición activa que muestra la web.

Figura 14

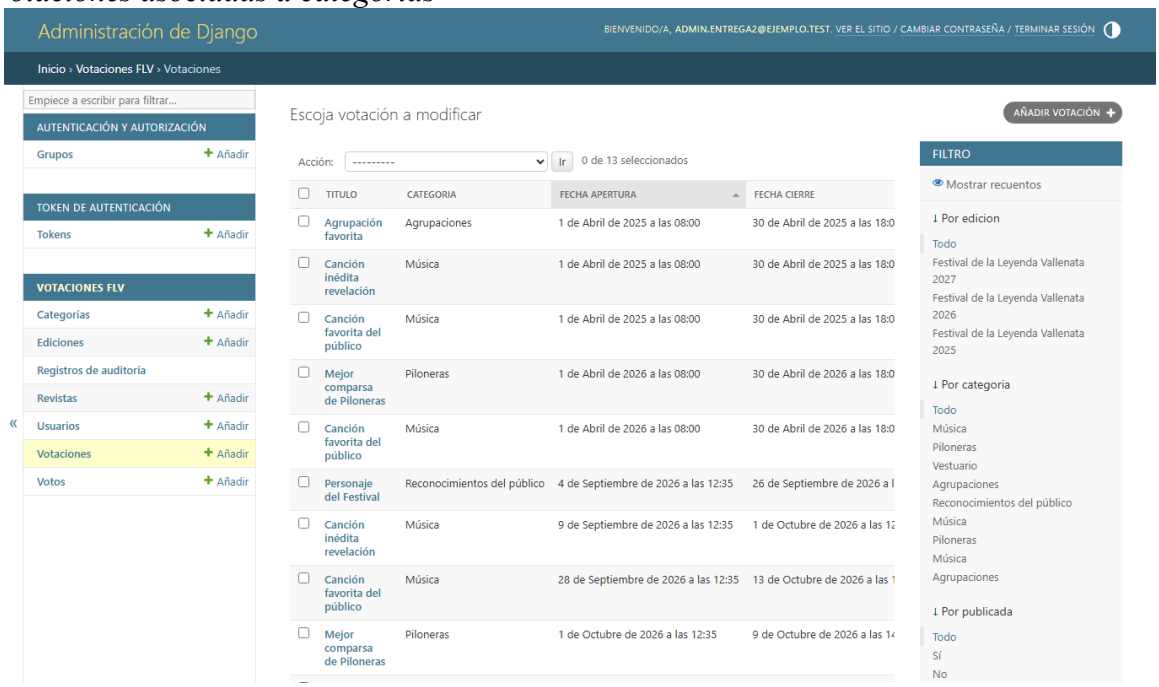
Categorías vinculadas a cada edición



Nota. Cada categoría pertenece a una edición (columna Edición) y se puede filtrar por ella. Las categorías son ilustrativas y siguen pendientes de validación con la Fundación.

Figura 15

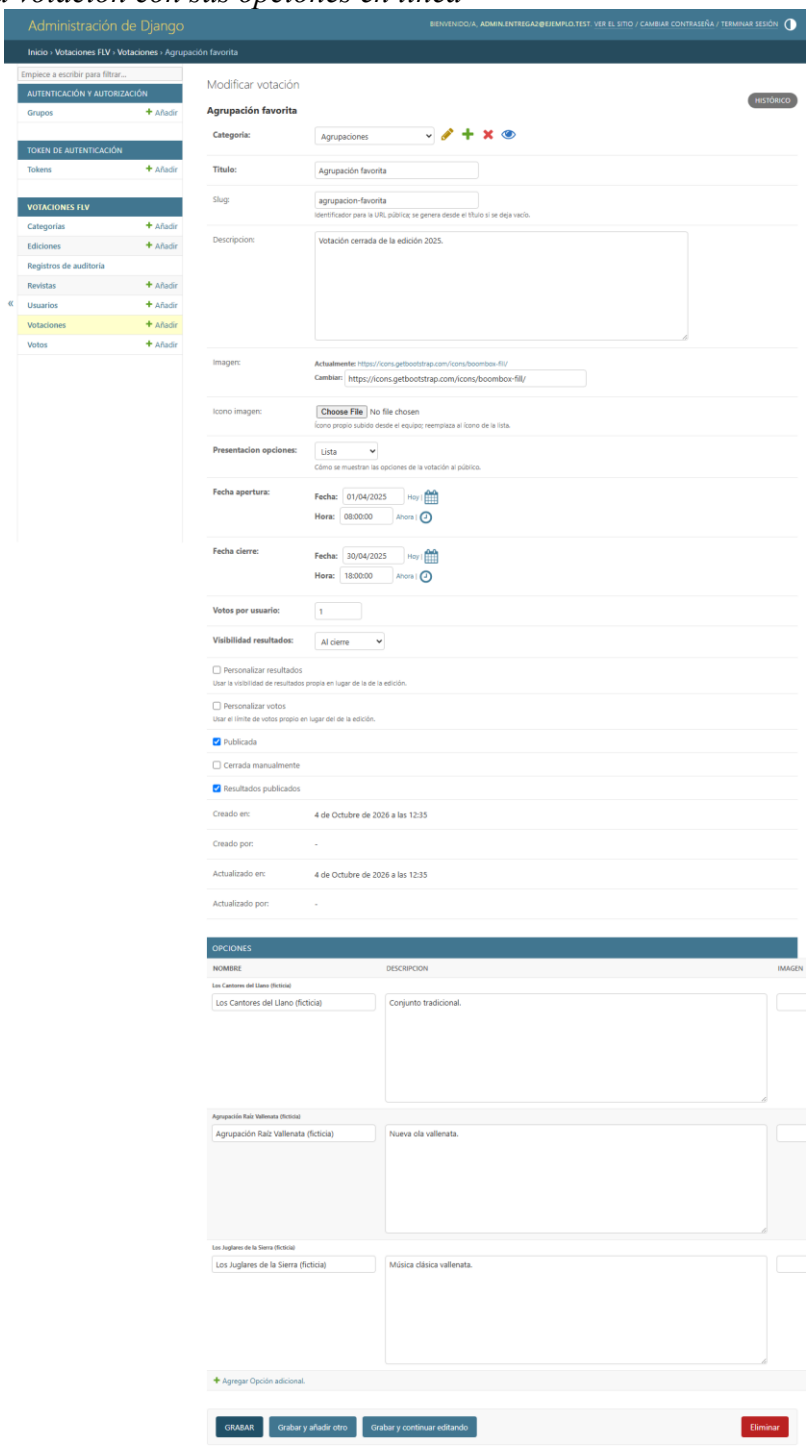
Votaciones asociadas a categorías



Nota. Cada votación muestra su categoría, fechas de apertura y cierre, si está publicada y su estado calculado (programada, abierta o cerrada).

Figura 16

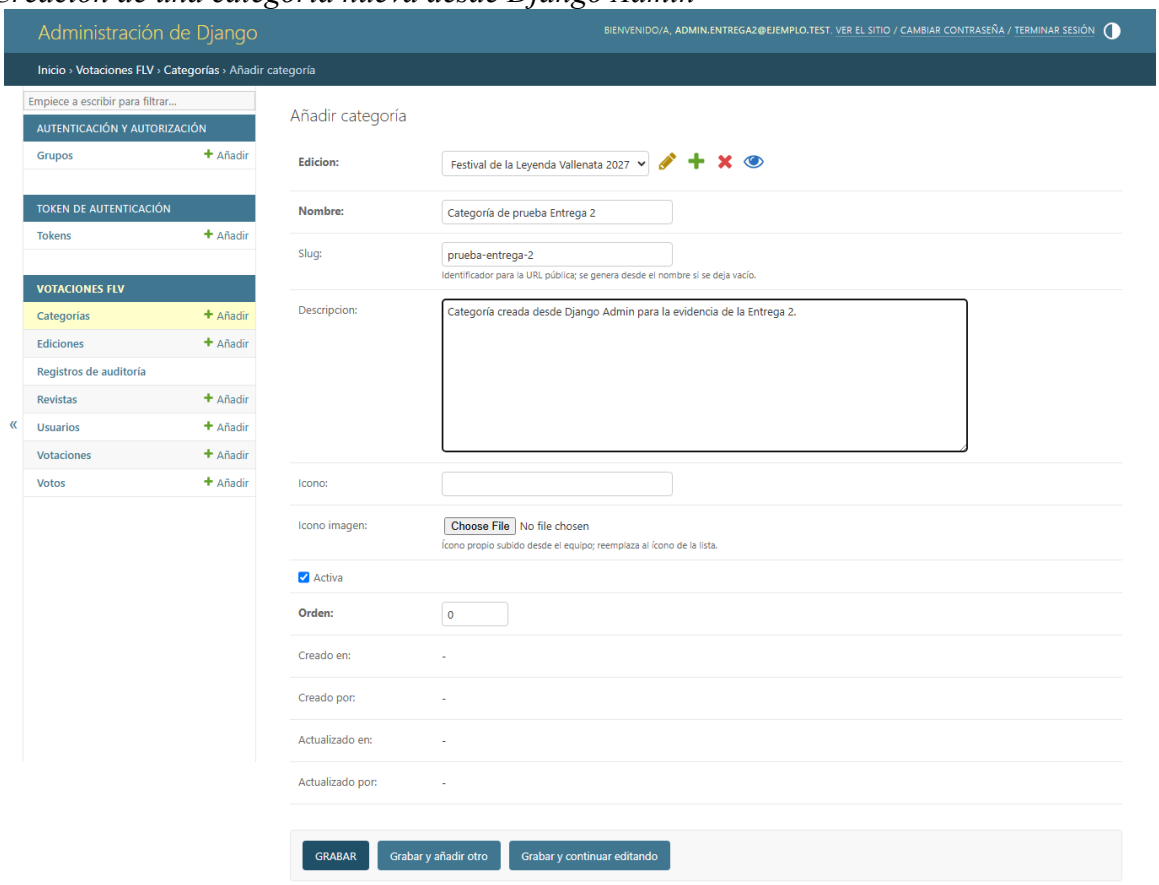
Edición de una votación con sus opciones en línea



Nota. Desde la misma pantalla se configura la votación (fechas, límite de votos, visibilidad de resultados) y se administran sus opciones participantes, lo que materializa la relación uno a muchos entre Votación y Opción. La captura se recortó para mostrar solo el nombre y la descripción de cada opción.

Figura 17

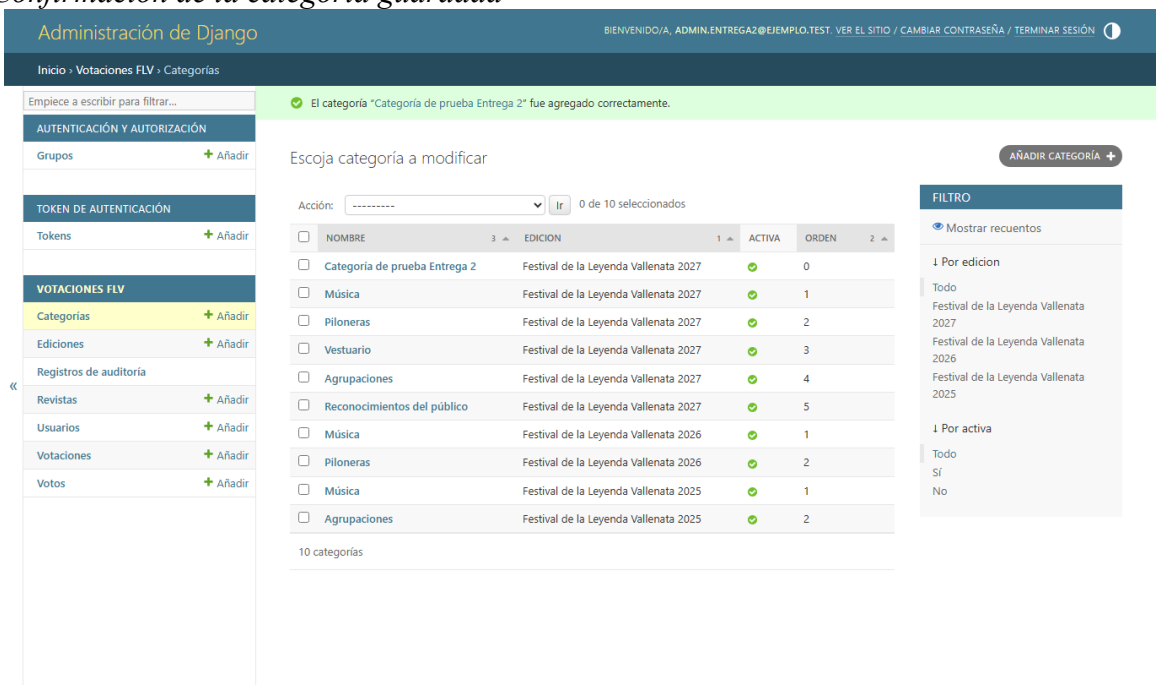
Creación de una categoría nueva desde Django Admin



Nota. Se registra la categoría «Categoría de prueba Entrega 2» en la edición 2027 para comprobar que se guarda en MySQL.

Figura 18

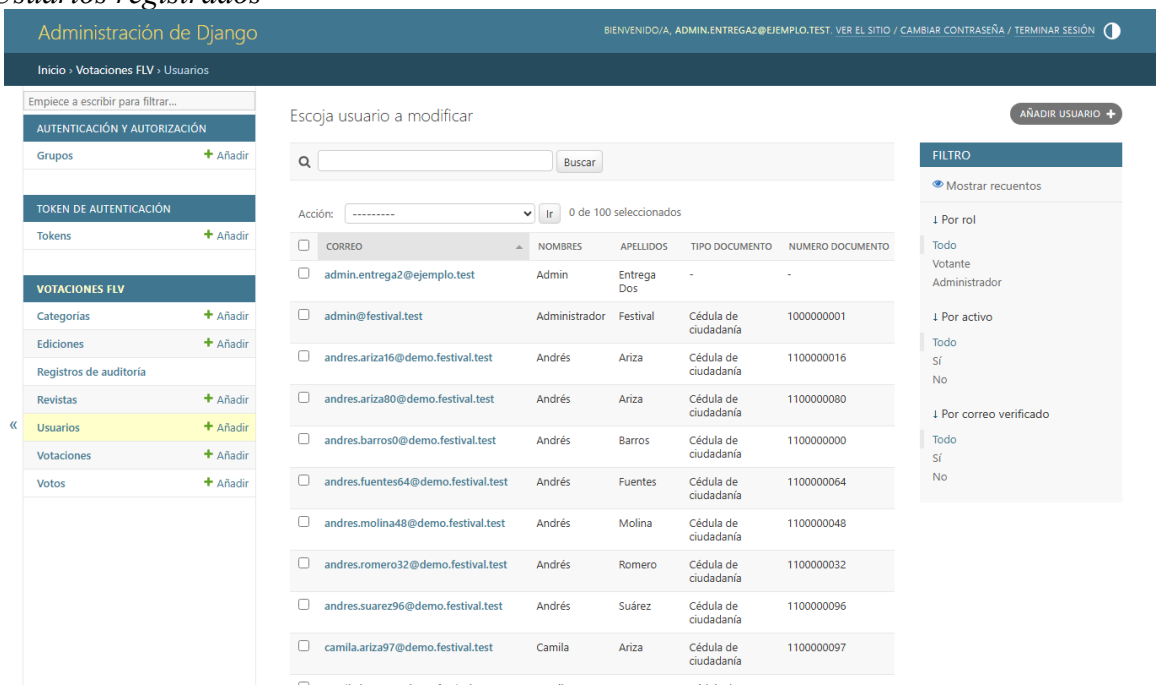
Confirmación de la categoría guardada



Nota. Django Admin confirma que el registro se guardó en MySQL y la nueva categoría aparece en el listado.

Figura 19

Usuarios registrados

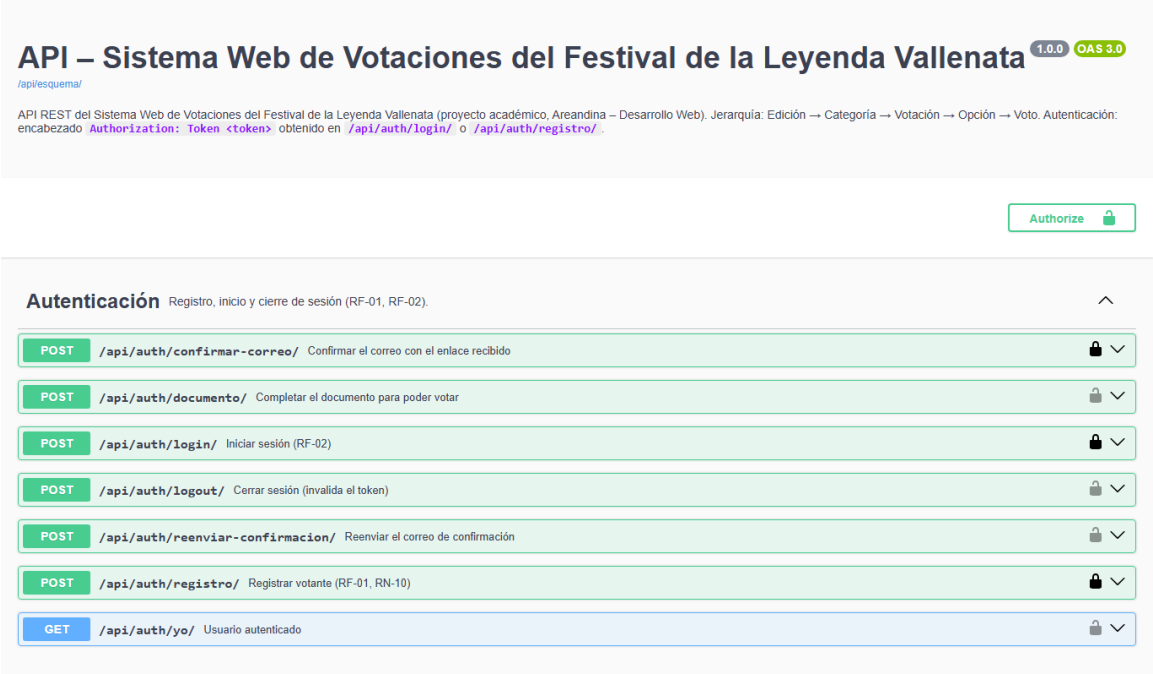


Nota. Listado del modelo de usuario propio. Los datos son ficticios y las contraseñas se guardan cifradas (hash), por lo que nunca se muestran.

Vistas y documentación de la API

Figura 20

Documentación de las vistas de la API (Swagger)

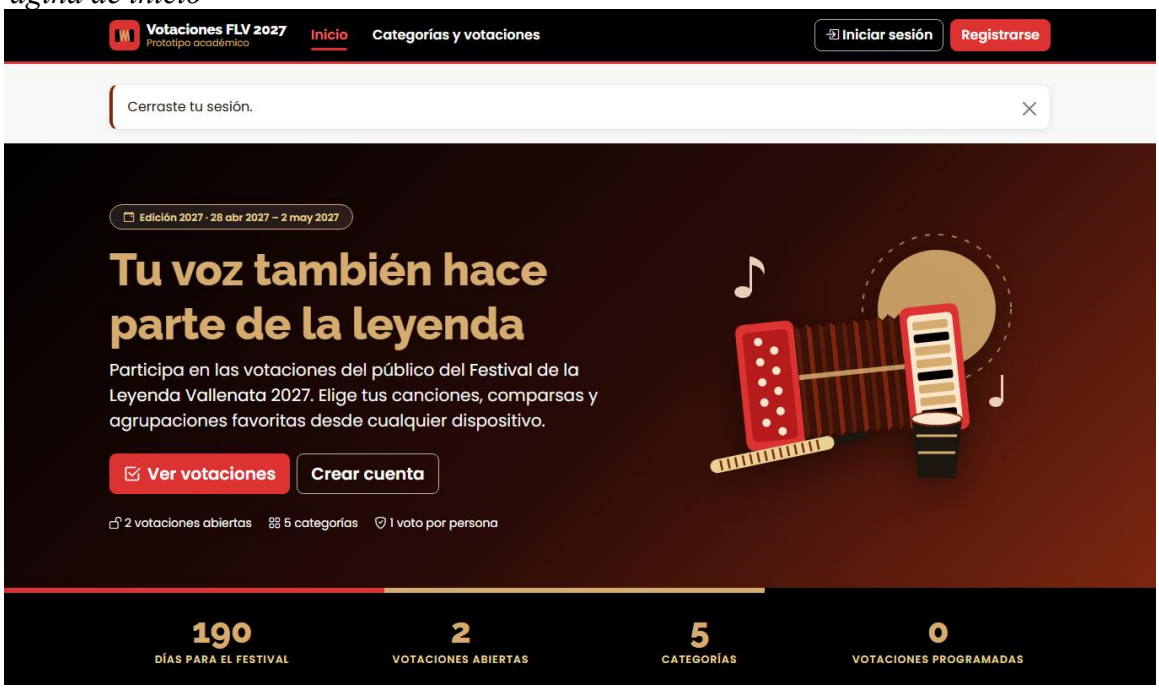


Nota. Las vistas de Django quedan documentadas en /api/docs/ con el esquema OpenAPI generado automáticamente; se ven los grupos de autenticación, consulta pública y administración.

Interfaz pública con datos de MySQL

Figura 21

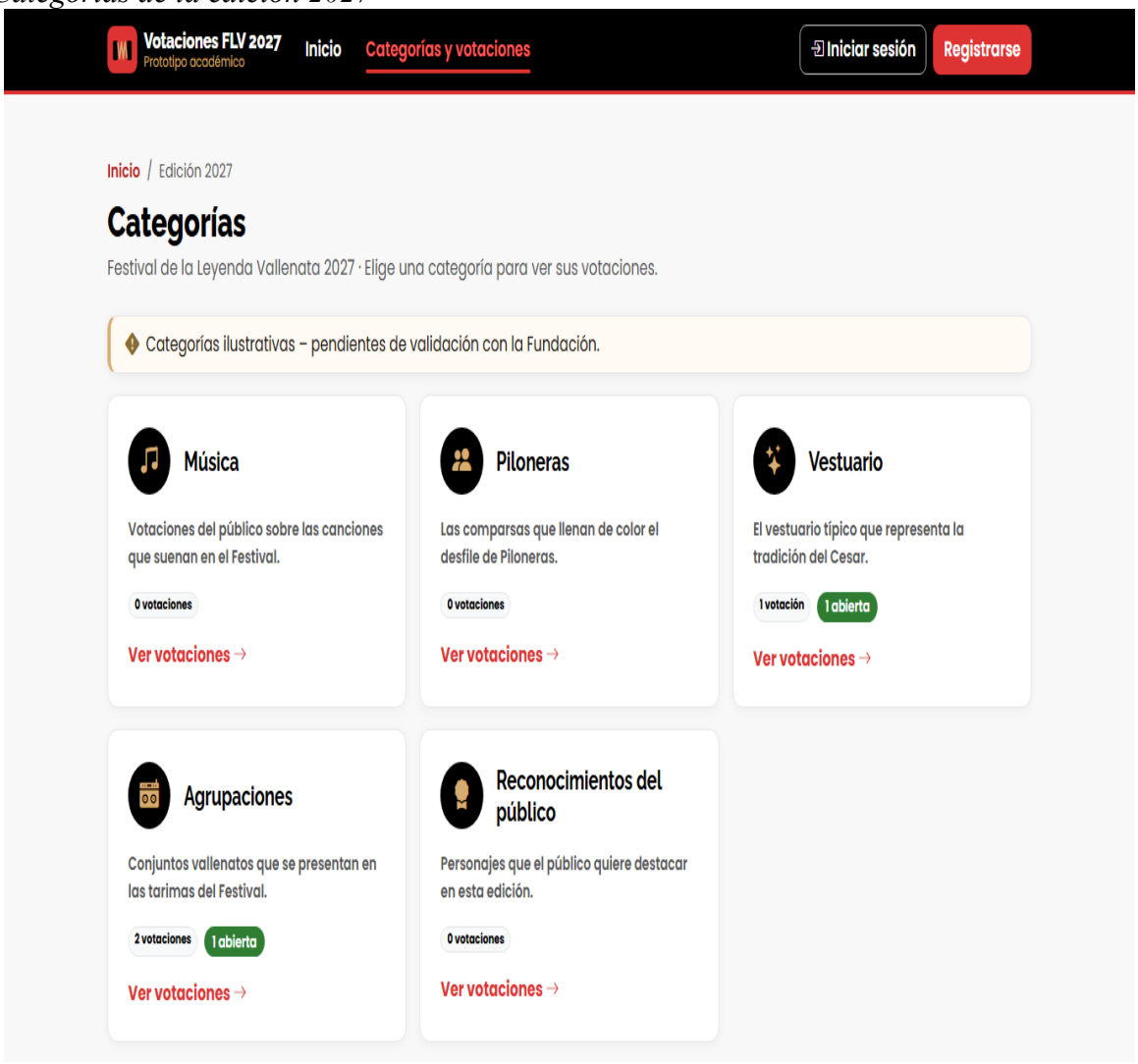
Página de inicio



Nota. La vista inicio (portal/vistas/publico.py) consulta la edición activa y sus votaciones abiertas con el ORM y la plantilla inicio.html las muestra.

Figura 22

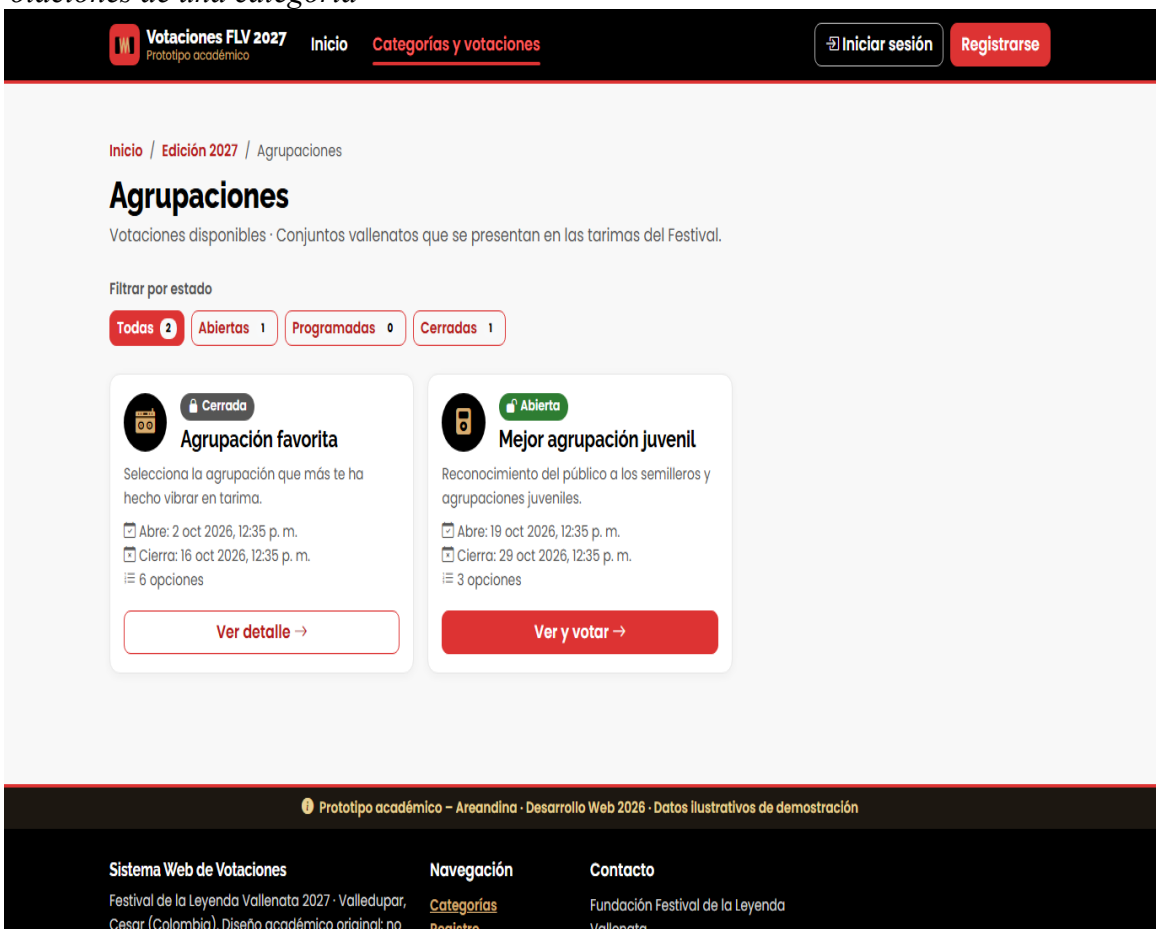
Categorías de la edición 2027



Nota. La plantilla recorre las categorías activas de la edición; el número de votaciones de cada una lo calcula la vista con una consulta a MySQL.

Figura 23

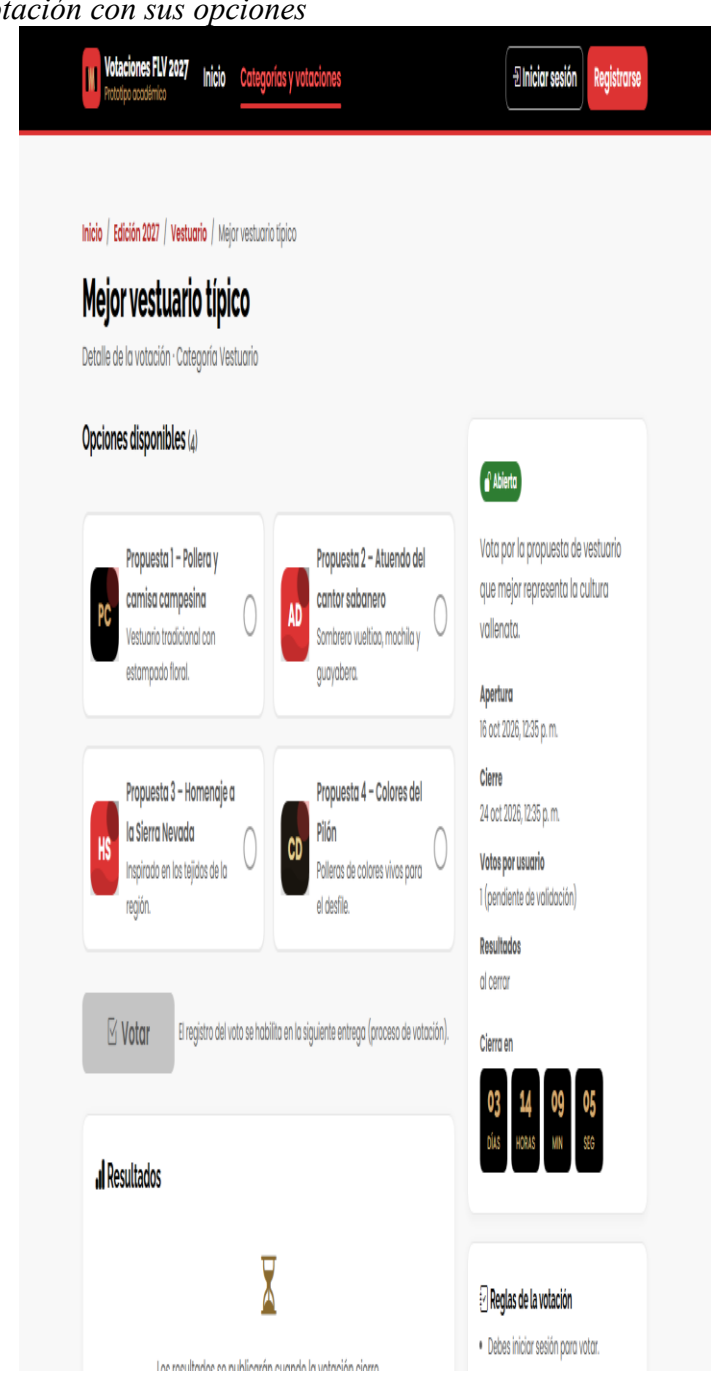
Votaciones de una categoría



Nota. Al elegir una categoría se listan sus votaciones con su estado y sus fechas; el filtro por estado usa parámetros de la URL (?estado=abierta).

Figura 24

Detalle de una votación con sus opciones

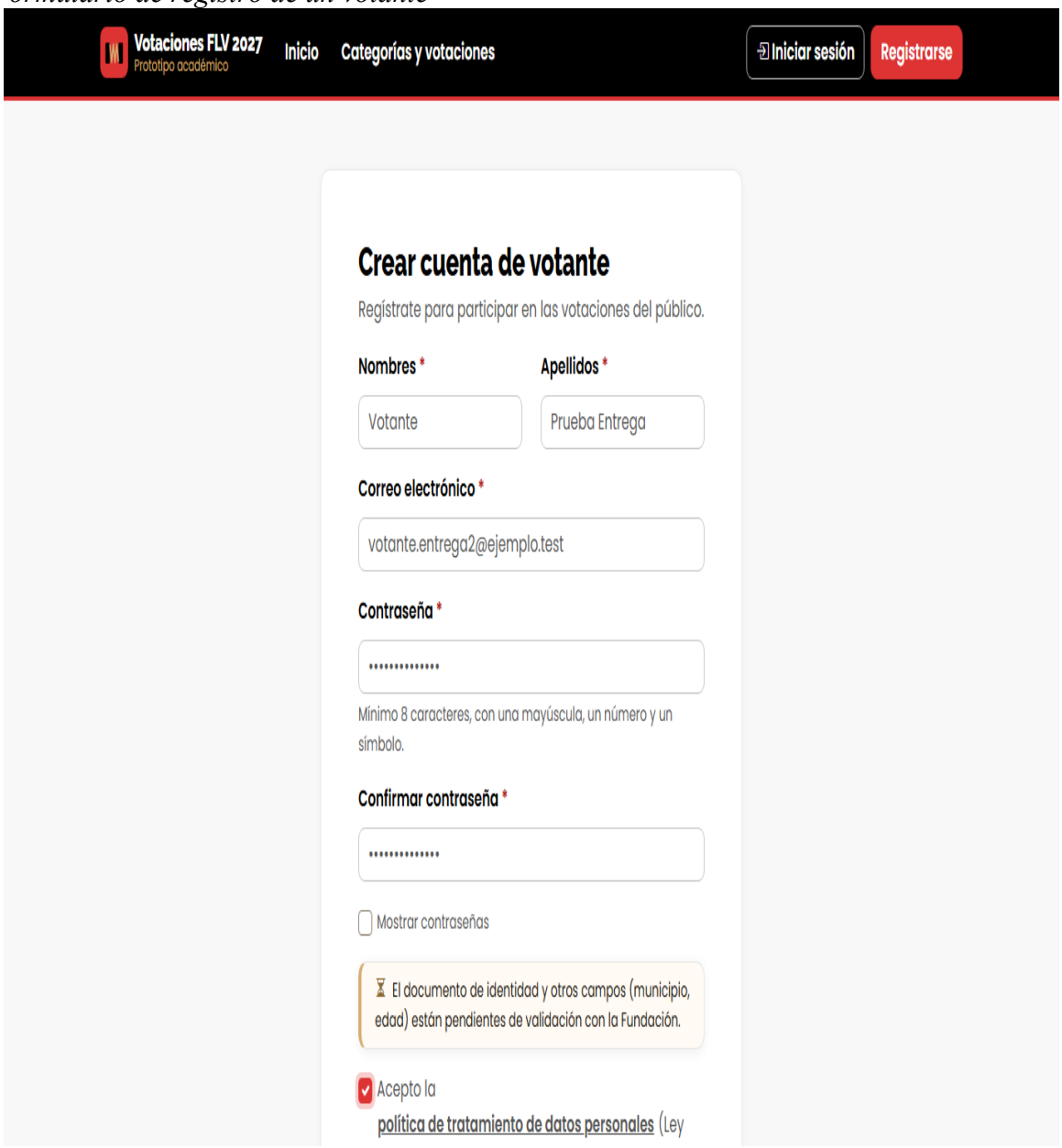


Nota. Muestra la votación y las opciones registradas en el panel. El registro definitivo del voto corresponde a la Entrega 3, por eso el botón «Votar» aparece deshabilitado.

Autenticación

Figura 25

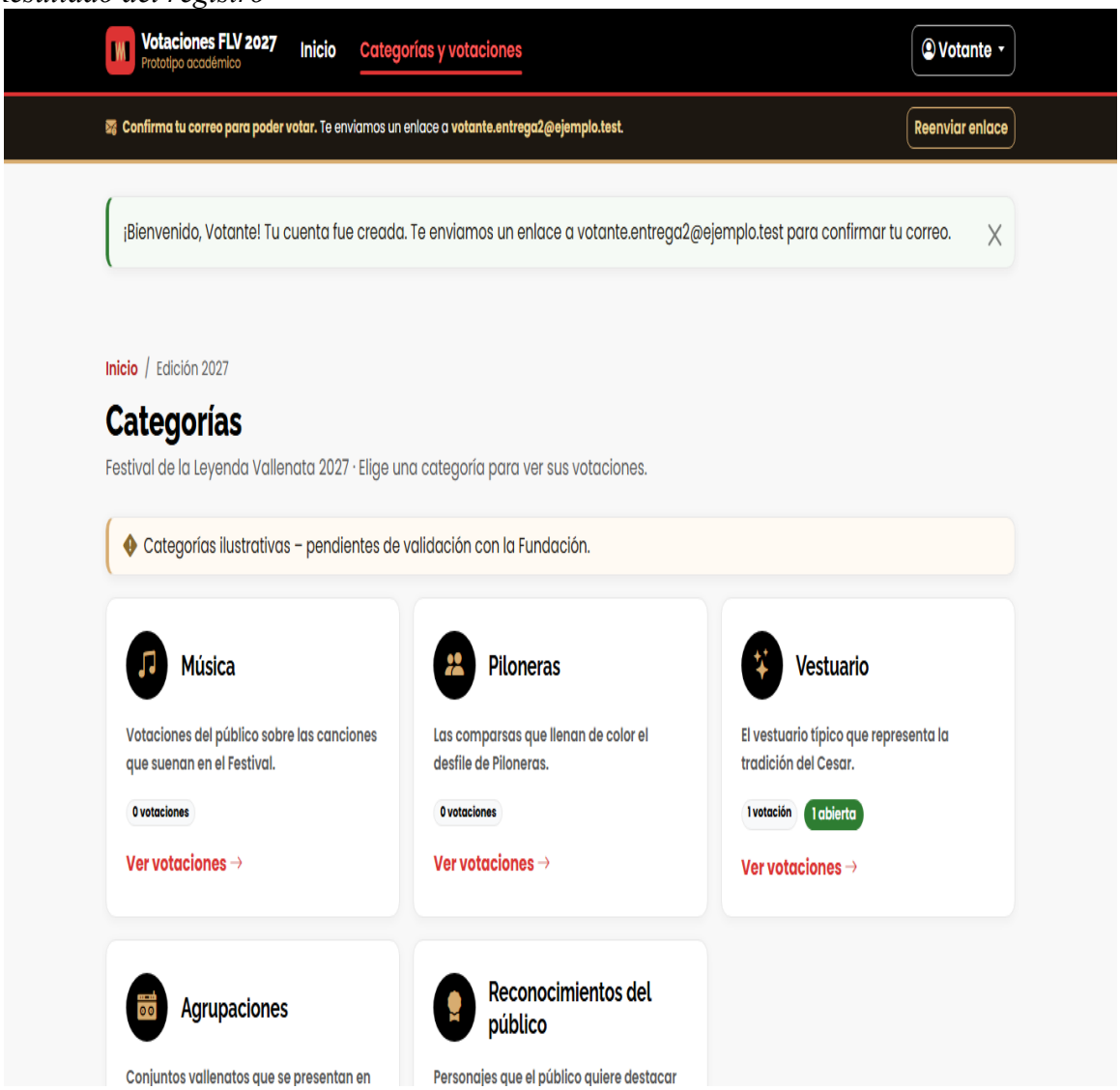
Formulario de registro de un votante



Nota. Formulario de Django (RegistroForm): nombres, apellidos, correo, contraseña y autorización de tratamiento de datos (Ley 1581 de 2012). Las contraseñas se ven ocultas.

Figura 26

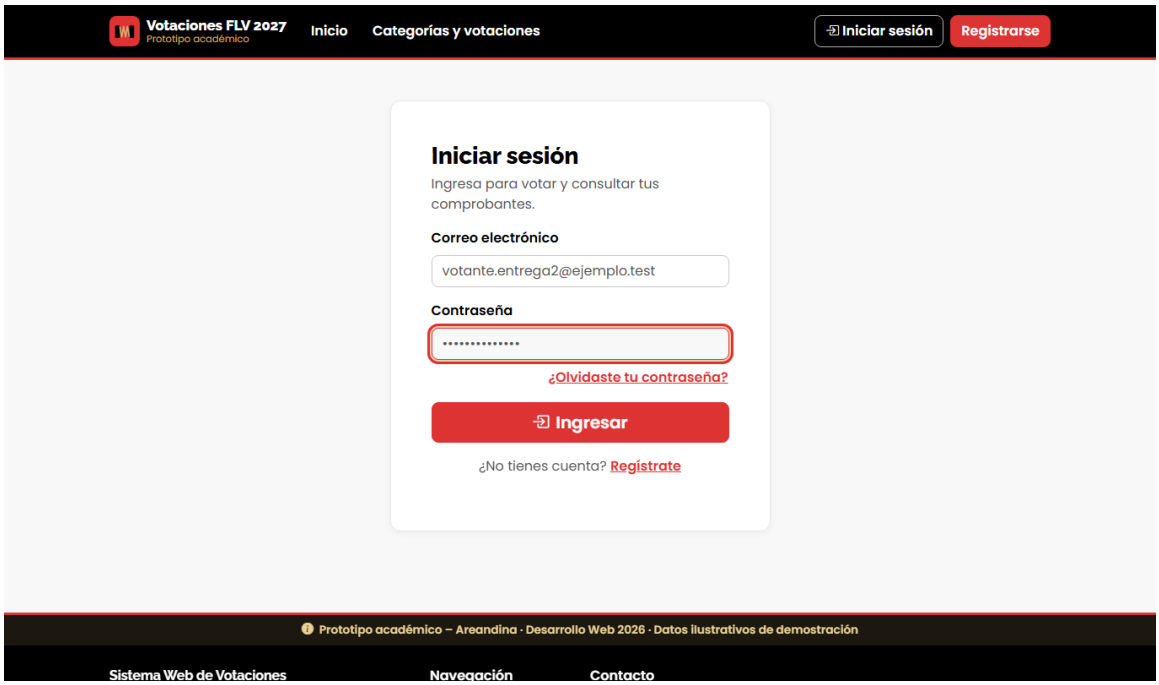
Resultado del registro



Nota. La cuenta queda creada y la sesión iniciada con login(); el mensaje confirma el registro y la barra superior pide confirmar el correo antes de votar (RN-18 de la Entrega 1).

Figura 27

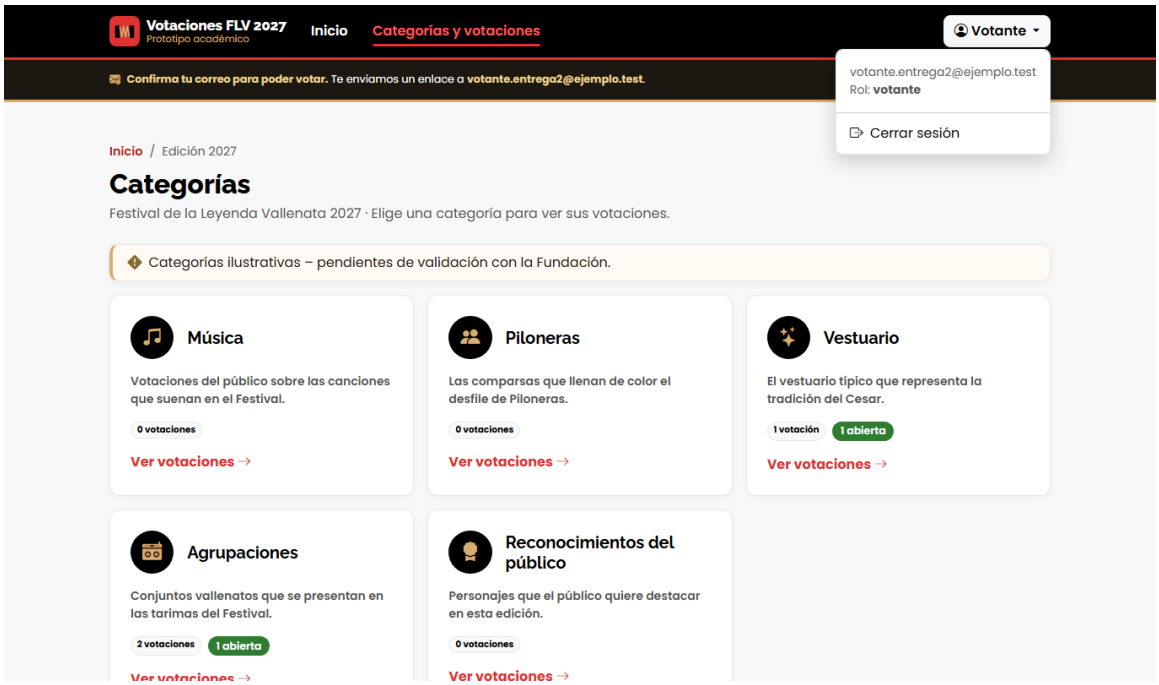
Inicio de sesión



Nota. Inicio de sesión con correo y contraseña de un votante de prueba; la vista valida con authenticate().

Figura 28

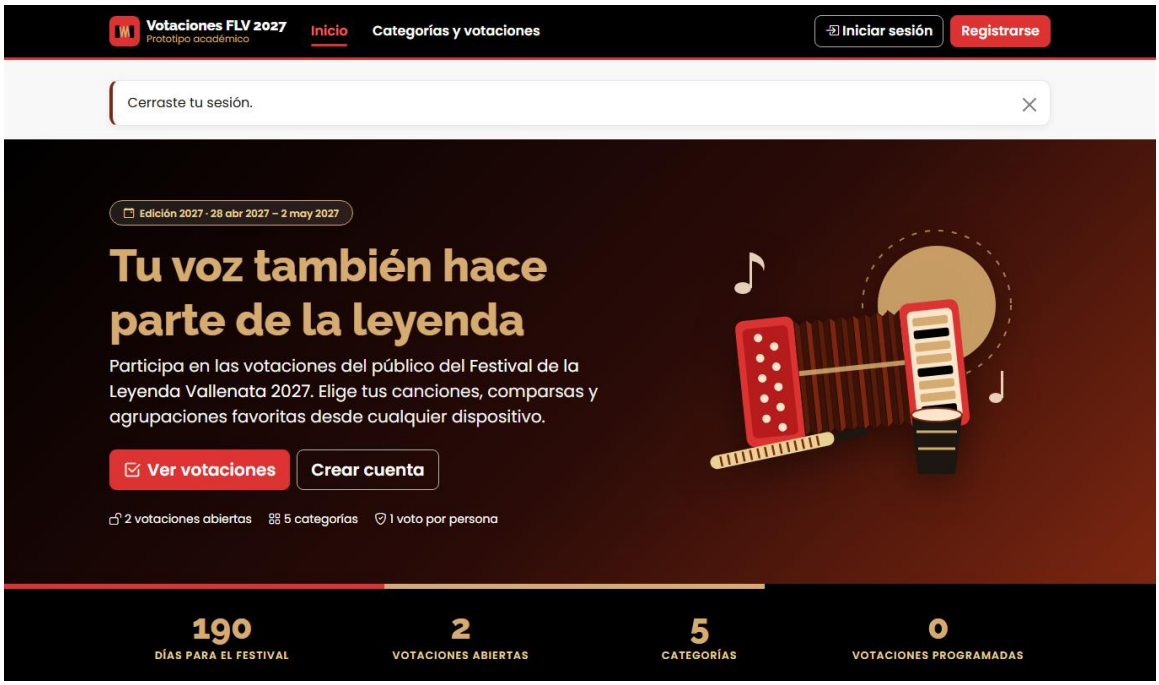
Sesión iniciada



Nota. Con la sesión de Django activa, la barra superior muestra el nombre del usuario y el menú con «Cerrar sesión».

Figura 29

Sesión cerrada

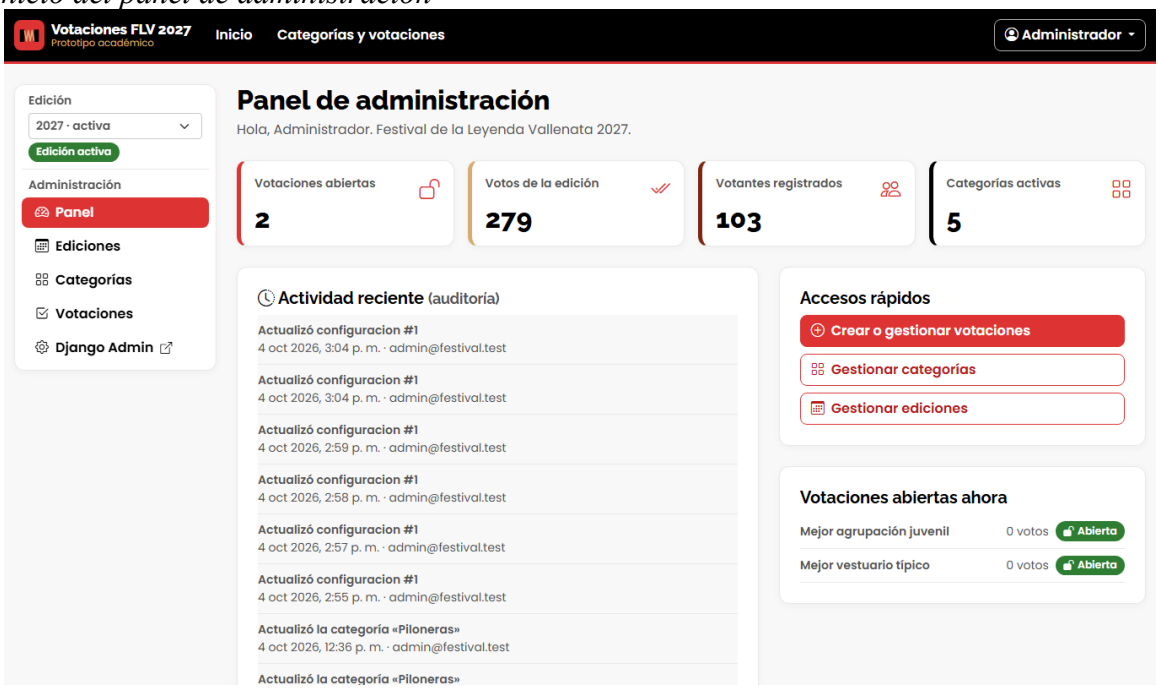


Nota. El cierre de sesión se envía por POST con token CSRF; logout() elimina la sesión y vuelven los botones «Iniciar sesión» y «Registrarse».

Panel de administración propio (CRUD con Django)

Figura 30

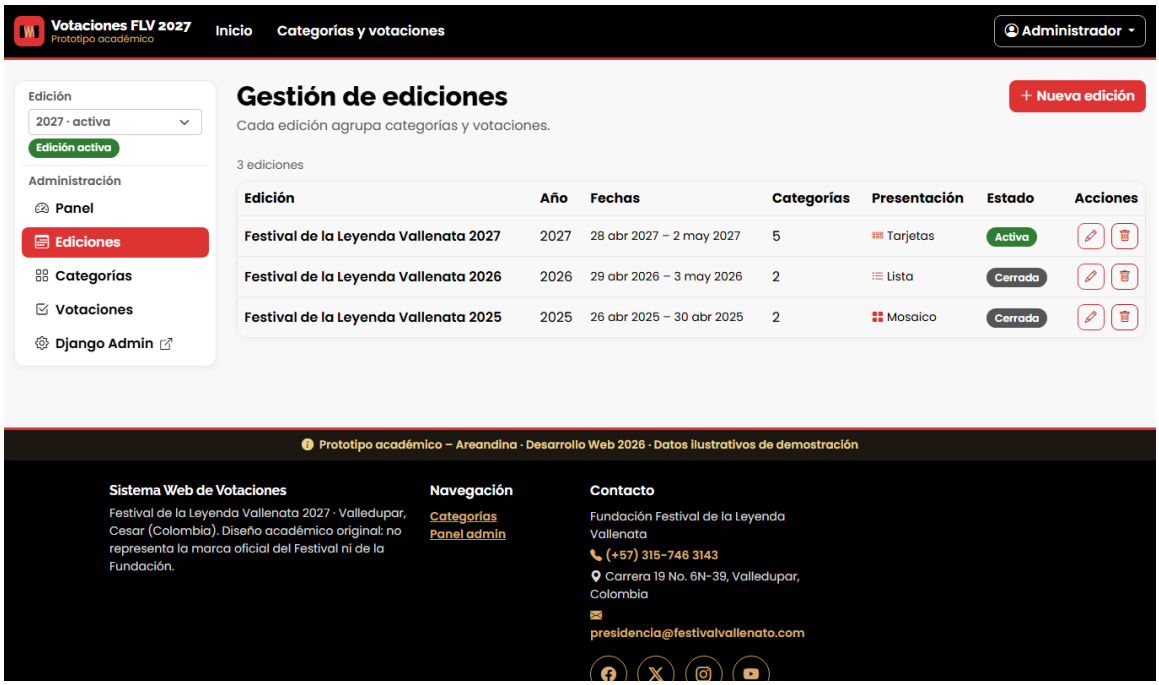
Inicio del panel de administración



Nota. Indicadores de la edición, actividad reciente (auditoría) y votaciones abiertas. Solo entra un usuario con rol de administrador.

Figura 31

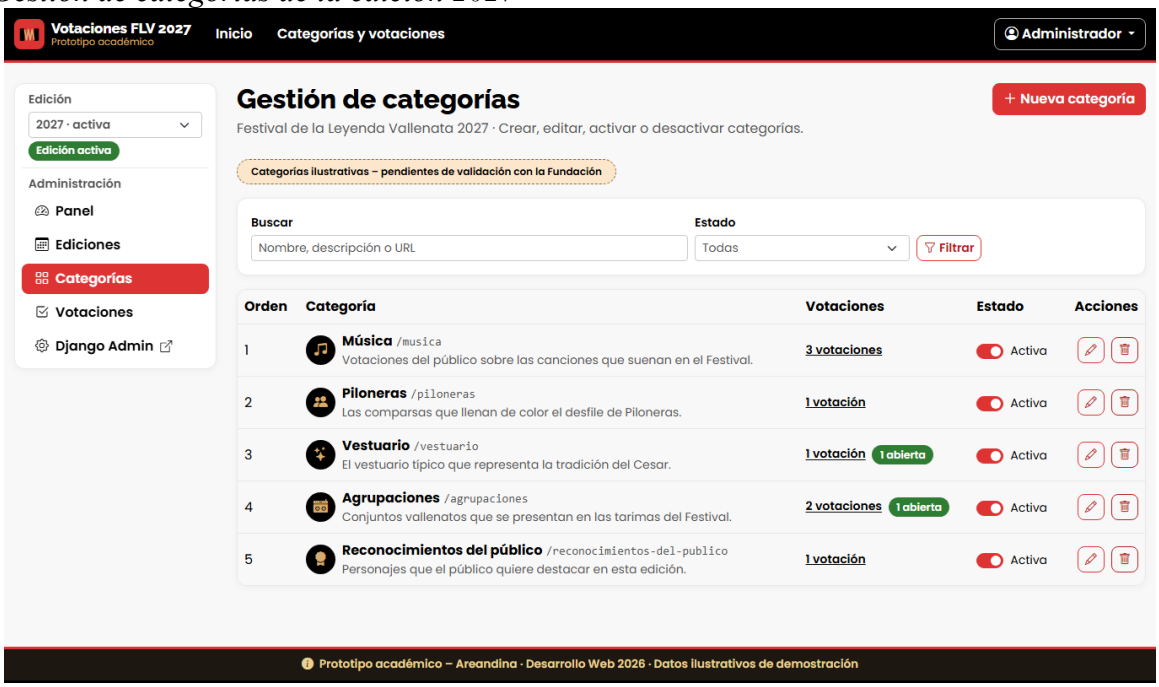
Gestión de ediciones



Nota. Listado de las ediciones guardadas en MySQL con sus fechas, categorías, presentación y estado; desde aquí se crean, editan y eliminan.

Figura 32

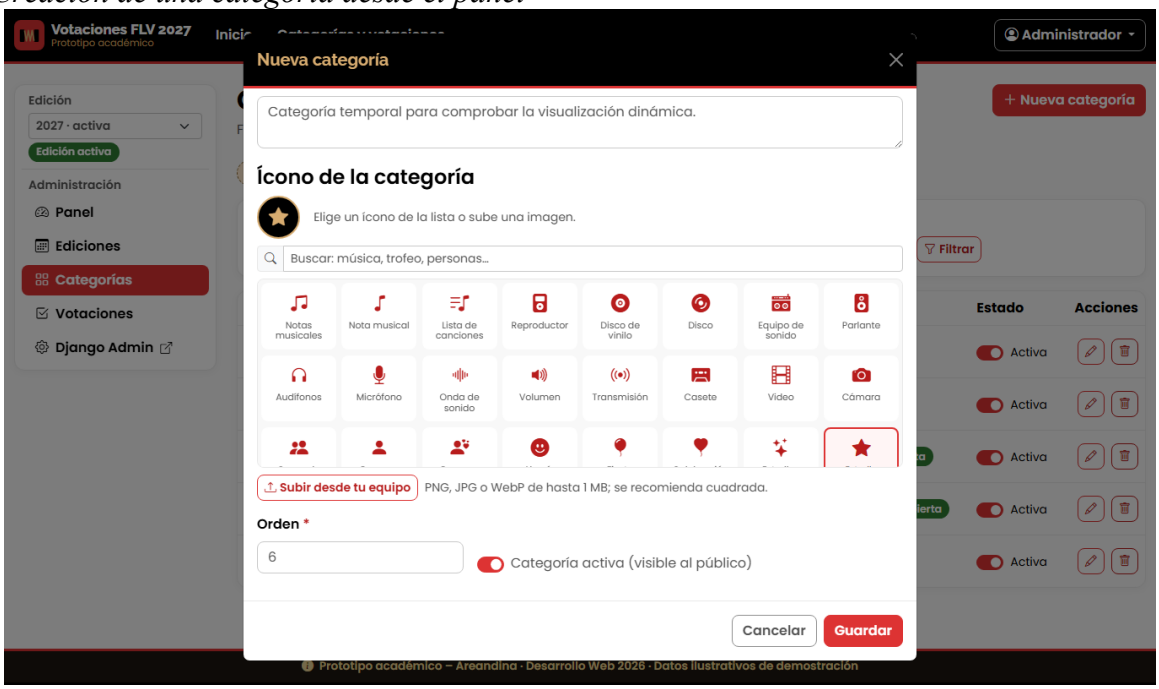
Gestión de categorías de la edición 2027



Nota. Cada categoría muestra su ruta pública, sus votaciones y un interruptor para activarla o desactivarla. El selector de la izquierda cambia la edición consultada.

Figura 33

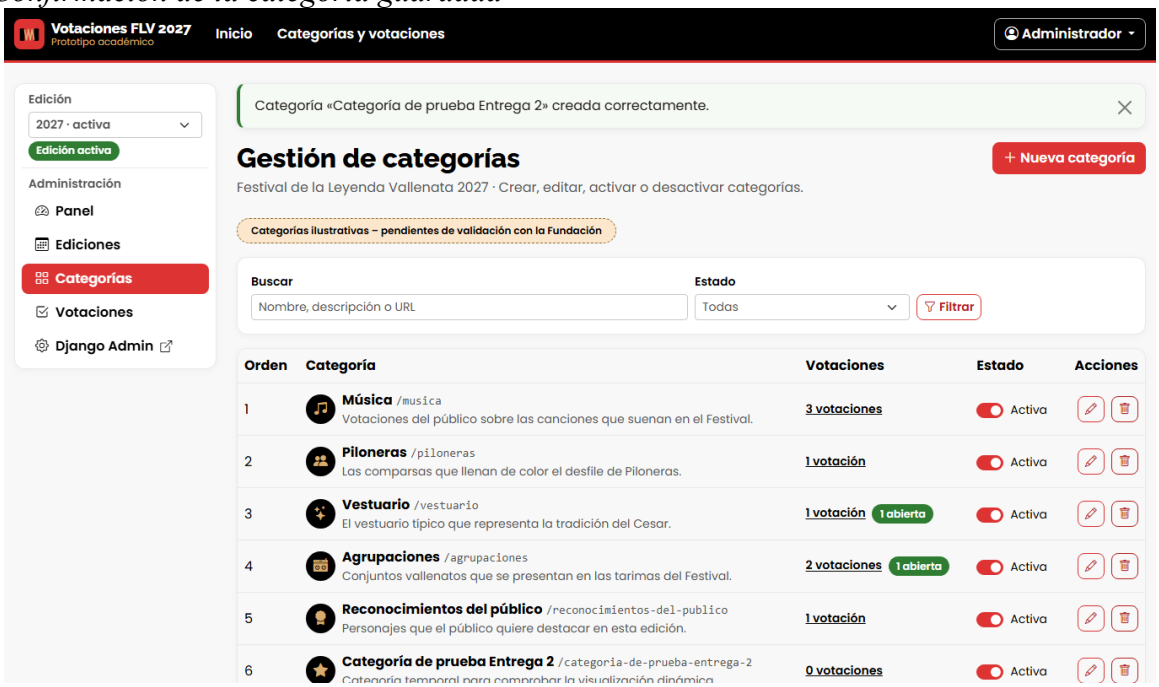
Creación de una categoría desde el panel



Nota. Formulario de Django en ventana modal: edición, nombre, descripción, ícono y orden. Se registra «Categoría de prueba Entrega 2».

Figura 34

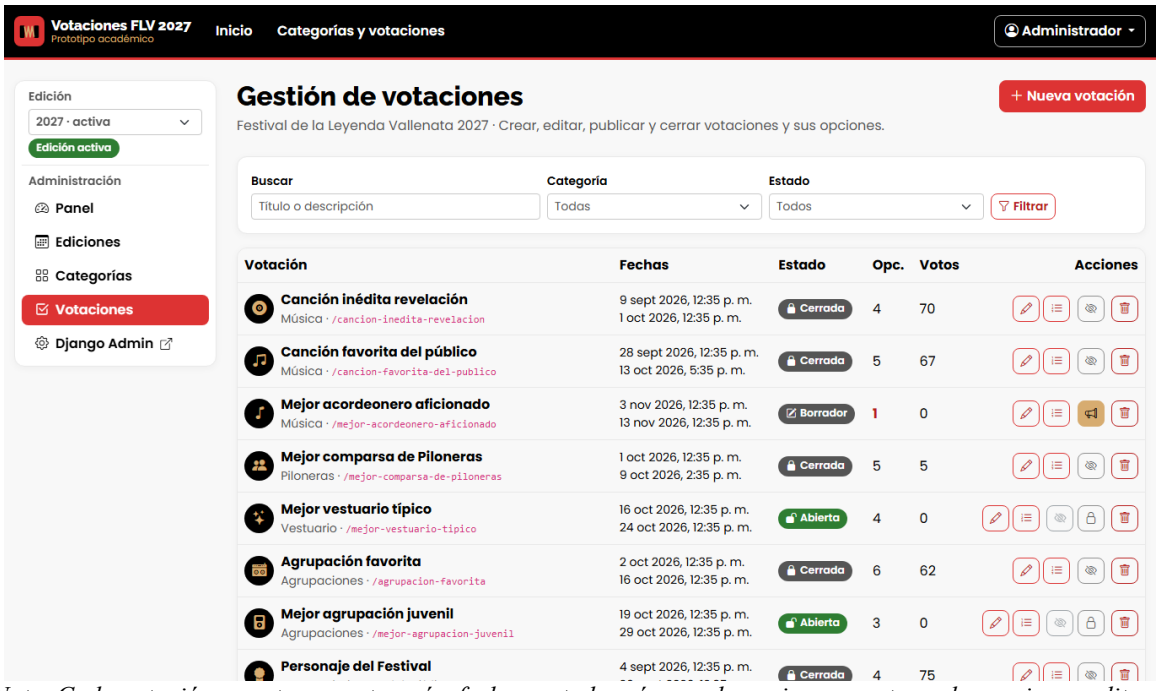
Confirmación de la categoría guardada



Nota. El mensaje confirma que el registro se guardó en MySQL y la categoría aparece en el listado; la acción queda en la auditoría.

Figura 35

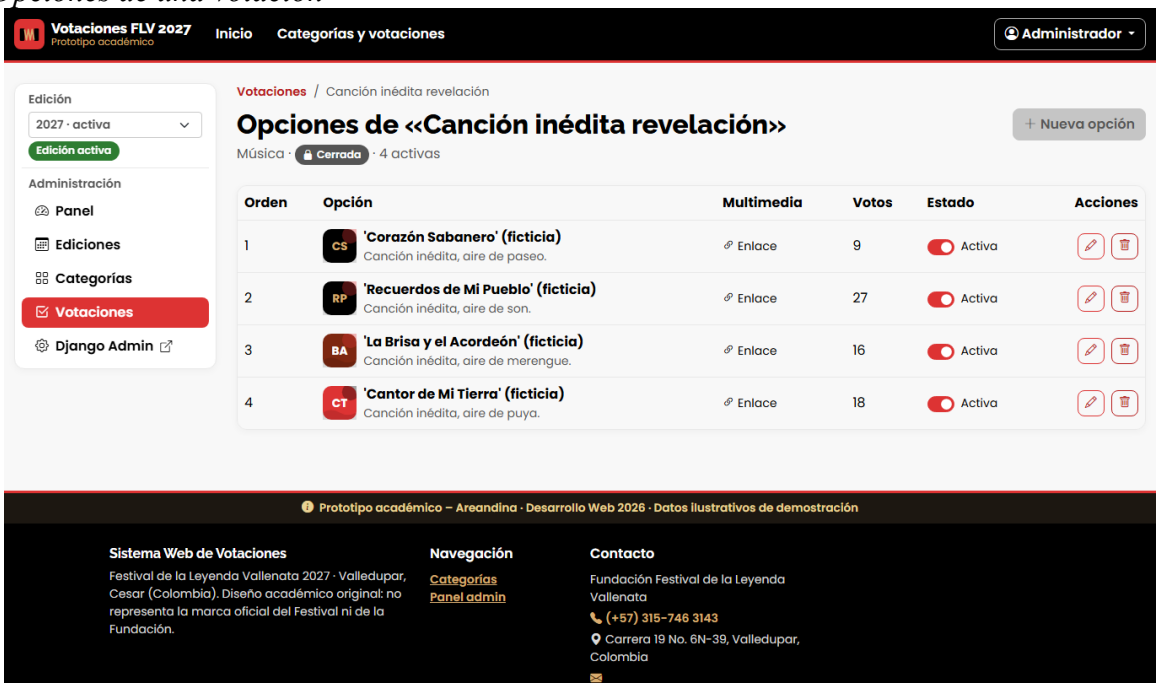
Gestión de votaciones



Nota. Cada votación muestra su categoría, fechas, estado, número de opciones y votos, y las acciones editar, opciones, publicar o despublicar, cerrar y eliminar.

Figura 36

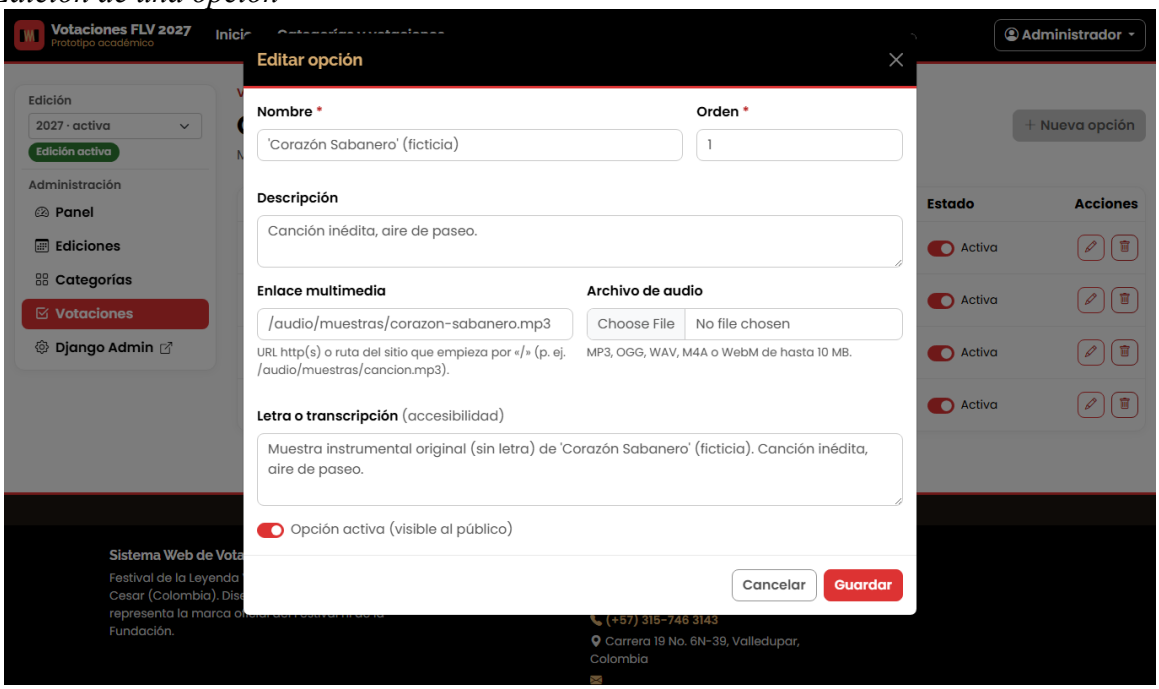
Opciones de una votación



Nota. Materializa la relación uno a muchos entre Votación y Opción. Como la votación está cerrada, no admite opciones nuevas.

Figura 37

Edición de una opción

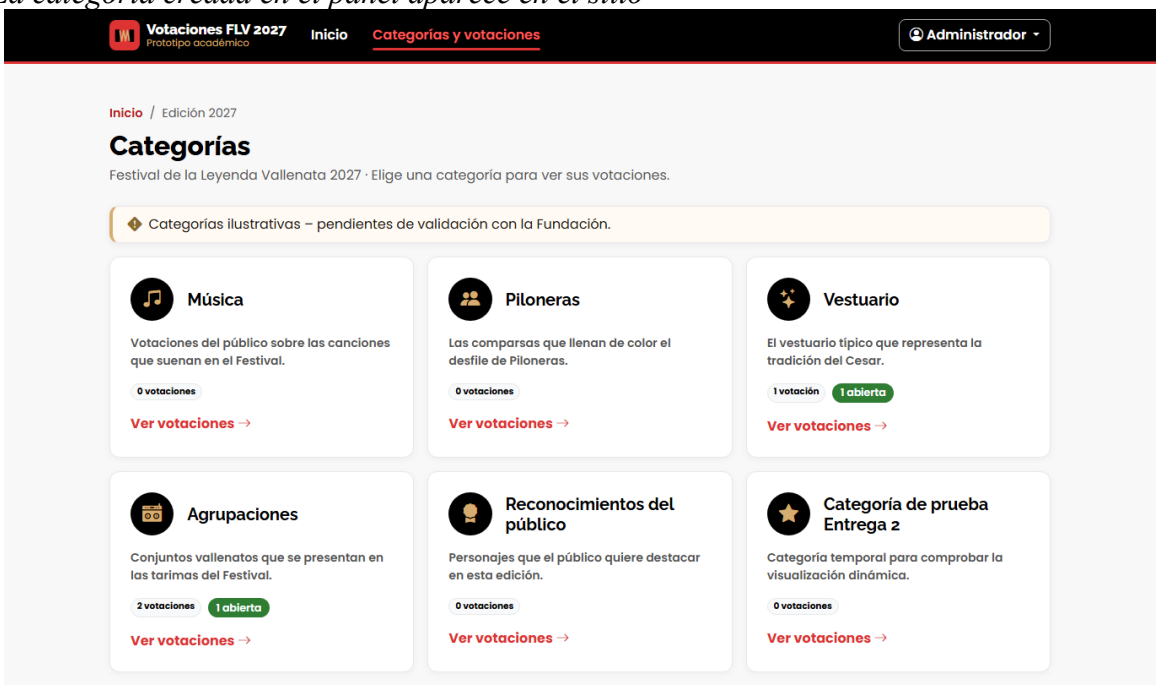


Nota. Formulario con nombre, descripción, enlace o archivo de audio y la letra o transcripción para accesibilidad.

Visualización dinámica

Figura 38

La categoría creada en el panel aparece en el sitio



Nota. Demuestra que la interfaz pública no tiene datos fijos: la categoría registrada en la Figura 33 se muestra de inmediato porque la vista la consulta en MySQL. Después de la prueba se borró.

Pruebas y resultados

Pruebas automatizadas

El proyecto tiene 219 pruebas automatizadas escritas con el módulo de pruebas de Django: 180 de la API y de las reglas de negocio y 39 del sitio construido con vistas y plantillas (páginas públicas, registro, sesión y CRUD del panel). Cada prueba crea una base de datos temporal, ejecuta el caso y la destruye al final, por lo que no afecta los datos reales. Las pruebas se ejecutan en cada cambio que se sube a GitHub y, para esta entrega, también contra MySQL 8.4.

Procedimiento: docker compose exec api python manage.py test

Tabla 5

Pruebas automatizadas por archivo

Archivo	Qué verifica	Pruebas
test_auth.py	Registro, inicio y cierre de sesión por la API, confirmación del correo, perfil	22
test_admin.py	Administración de ediciones, categorías, votaciones y opciones por la API; permisos	18
test_publico.py (API)	Consulta pública de ediciones, categorías, votaciones y opciones	14
test_integracion_web.py	Respuestas que consume la web React (formato y campos)	28
test_configuracion_edicion.py · test_votar.py	Configuración por edición y base del proceso de voto	17
test_participacion_integridad.py · test_seguridad.py	Reportes de integridad, permisos y cabeceras de seguridad	23
test_autoria.py · test_correo.py	Auditoría automática y correos de confirmación y comprobante	32
test_sitio.py, test_seo.py, test_marca.py, test_ruta_gestion.py	Contenido del sitio, mapa del sitio, marca y rutas	26
portal/tests/test_publico.py	Vistas y plantillas públicas: edición activa, categorías, filtros, detalle, borradores y resultados	12
portal/tests/test_cuentas.py	Registro con formulario de Django, inicio y cierre de sesión, redirecciones seguras, confirmación del correo	10
portal/tests/test_panel.py	Acceso al panel y CRUD de ediciones, categorías, votaciones y opciones con sus reglas	17
Total		219

Figura 39

Resultado de las pruebas contra MySQL

```
python manage.py test
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
…
----------------------------------------------------------------------
Ran 219 tests in 72.208s

OK
Destroying test database for alias 'default'...
```

Nota. Salida de consola tomada el 20 de octubre de 2026 en Docker con MySQL 8.4: 219 pruebas ejecutadas en 72 segundos, todas correctas (OK).

Pruebas funcionales

Tabla 6

Pruebas funcionales realizadas

Funcionalidad	Procedimiento	Resultado esperado	Resultado
Conexión con MySQL	Arrancar Docker y ejecutar showmigrations	Migraciones aplicadas en MySQL 8.4	Correcto (Figura 8)
Ingreso a Django Admin	Entrar con un superusuario de prueba	Se ven los modelos registrados	Correcto (Figura 12)
Ingreso al panel propio	Iniciar sesión como administrador	Se abre /panel con los indicadores	Correcto (Figura 30)
Crear categoría	Registrar una categoría en la edición 2027 desde el panel	Se guarda y aparece en el listado	Correcto (Figuras 33–34)
Opciones de una votación	Abrir las opciones de una votación y editar una	Se ven y editan sus opciones	Correcto (Figuras 36–37)
Visualización dinámica	Abrir el sitio después de crear la categoría	El sitio la muestra sin cambiar código	Correcto (Figura 38)
Registro	Llenar el formulario de registro	Cuenta creada y aviso de confirmar correo	Correcto (Figuras 25–26)
Inicio de sesión	Ingresar correo y contraseña válidos	Se muestra el usuario en la barra	Correcto (Figuras 27–28)
Cierre de sesión	Elegir «Cerrar sesión»	Vuelven los botones de ingreso	Correcto (Figura 29)

AREANDINA

Repositorio

El código fuente está en un repositorio público de GitHub. El archivo README explica la estructura del proyecto y los pasos de instalación y ejecución con Docker y MySQL: clonar el repositorio, ejecutar docker compose up -d --build, cargar los datos de prueba con python manage.py cargar_demo y crear un superusuario con python manage.py createsuperuser. El sitio Django queda en http://localhost:8096/ y el panel en /panel.

Repositorio: https://github.com/juandiegows/votaciones-festival-vallenato

Instalación y ejecución: https://github.com/juandiegows/votaciones-festival-vallenato#instalación-y-ejecución-django--mysql

Versión publicada: https://votaciones.juandiegows.com

Demostración sin servidor (GitHub Pages): https://juandiegows.github.io/votaciones-festival-vallenato/

AREANDINA

Video demostrativo

El video muestra la estructura del proyecto Django, la conexión con MySQL por medio del adaptador, los modelos y las migraciones, el CRUD de ediciones, categorías, votaciones y opciones en el panel, la visualización dinámica de los datos en el sitio, el registro, el inicio y el cierre de sesión, y la ejecución de las pruebas. Cada integrante presenta una parte.

Enlace al video: [PENDIENTE: pegar el enlace al video]

Conclusiones y distribución del trabajo

Aprendizajes

• Patrón MVT. Separar modelos, vistas y plantillas permitió construir el sitio y el panel con el mismo diseño de la marca y probar cada vista de forma independiente con el cliente de pruebas de Django.

• Integridad y reglas en un solo lugar. Las restricciones de la base de datos y la capa de servicios hacen que el panel, Django Admin y la API apliquen las mismas reglas (por ejemplo, no publicar una votación con menos de dos opciones).

• Adaptadores. La base de datos (MySQL o SQLite) y el correo (ZeptoMail o SMTP) se cambian con una variable de entorno, sin tocar el código, y ninguna categoría ni votación está escrita en el código.

Dificultades

• **Migraciones.** Al empezar se unieron varias migraciones en una sola inicial para que MySQL y SQLite partieran del mismo esquema.

• **Permisos de MySQL.** Las pruebas contra MySQL requieren que el usuario de la base tenga permiso para crear la base de pruebas; quedó documentado en el README.

• **Requerimientos del cliente.** Las categorías y reglas definitivas siguen pendientes de validación con la Fundación, por lo que los datos de prueba son ilustrativos.

Pendientes para la siguiente entrega

• Registro definitivo del voto y control de votos duplicados.

• Reglas completas de elegibilidad y verificación del votante.

• Publicación final de resultados, auditoría avanzada y pruebas de carga.

• Respuestas de la Fundación a las preguntas P-01 a P-17 de la Entrega 1.

Distribución del trabajo

Tabla 7

Responsabilidades de cada integrante en la Entrega 2

Integrante	Responsabilidades
Juan Mejía Maestre	Coordinación del equipo; app portal: estilos de la marca, vistas y plantillas públicas, registro e inicio de sesión; Docker y revisión de coherencia.
Sebastián Bautista Martínez	Proyecto Django y adaptador de MySQL; modelos, relaciones y migraciones; panel de administración (CRUD de ediciones, categorías, votaciones y opciones).

Integrante	Responsabilidades
María Labarca Briceño	Formularios de registro, sesión y administración; pruebas automatizadas del sitio; documentación de la arquitectura, capturas e informe.

Nota. Elaboración propia a partir de los aportes registrados en el historial del repositorio de GitHub. Todos los integrantes revisaron el documento completo y pueden explicar cualquier parte de la solución.

Referencias

Django Software Foundation. (2025). Django documentation (versión 5.2). https://docs.djangoproject.com/en/5.2/

Django Software Foundation. (2025). The Django admin site. https://docs.djangoproject.com/en/5.2/ref/contrib/admin/

Django Software Foundation. (2025). Customizing authentication in Django. https://docs.djangoproject.com/en/5.2/topics/auth/customizing/

Django Software Foundation. (2025). Migrations. https://docs.djangoproject.com/en/5.2/topics/migrations/

Encode OSS. (2025). Django REST framework: Authentication. https://www.django-rest-framework.org/api-guide/authentication/

Oracle Corporation. (2025). MySQL 8.4 reference manual. https://dev.mysql.com/doc/refman/8.4/en/

Bootstrap Team. (2025). Bootstrap 5.3 documentation. https://getbootstrap.com/docs/5.3/

Docker Inc. (2025). Docker Compose overview. https://docs.docker.com/compose/

Congreso de Colombia. (2012). Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981